

LAPORAN PROPOSAL
PREDIKSI ADOPSI ARTIFICIAL INTELLIGENCE PADA DEVELOPER
MENGGUNAKAN LOGISTIC REGRESSION, RANDOM FOREST, DAN
CATBOOST BERDASARKAN STACK OVERFLOW DEVELOPER SURVEY 2025.



Dosen Pengampu:

Dr. Rahmat Budiarsa, S.Kom.

Disusun Oleh:

- Ilham Firmansyah (20240801102)
- Luqman Syahreno (20240801016)
- Siti Ahsanu Nadiyya Rizal (20240801155)
- Aidah Ruhiyah Ramadhani (20240801142)
- Muhammad Aditya (20240801234)

Mata Kuliah:

Machine Learning (CR001)

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS ESA UNGGUL
2026

DAFTAR ISI

DAFTAR ISI.....	ii
DAFTAR GAMBAR.....	vi
DAFTAR TABEL.....	vii
BAB I PENDAHULUAN.....	8
1.1 Latar Belakang.....	8
1.2 Rumusan Masalah.....	11
1.3 Batasan Masalah	12
1.4 Tujuan Penelitian	12
BAB II METODOLOGI PENELITIAN.....	13
2.1 Desain Penelitian	13
2.2 Perangkat dan Lingkungan Penelitian	13
2.3 Dataset Penelitian.....	14
2.3.1 Sumber dan karakteristik dataset	14
2.3.2 Jumlah data pada setiap skenario	14
2.4 Contoh Data Asli.....	15
2.5 Variabel Penelitian	16
2.5.1 Fitur prediktor	16
2.5.2 Target klasifikasi	18
2.6 Alur Penelitian	18
2.7 Preprocessing Data.....	21
2.7.1 Pemeriksaan kolom.....	21
2.7.2 Pemilihan dan penggantian nama kolom	21
2.7.3 Transformasi usia	21
2.7.4 Transformasi pengalaman coding	21

2.7.5 Pembentukan target biner.....	22
2.7.6 Pembagian data training dan testing	22
2.7.7 Pemadatan data multi-value	22
2.7.8 Penanganan missing value	22
2.7.9 Penanganan nilai ekstrem	23
2.7.10 Transformasi kompensasi.....	23
2.7.11 Encoding data kategorikal.....	23
2.7.12 Standardisasi fitur	23
2.8 Penanganan Ketidakseimbangan Kelas	24
2.8.1 SMOTENC	24
2.8.2 Class weighting.....	24
2.8.3 Random under-sampling.....	24
2.9 Skenario Pemodelan.....	25
2.10 Model Logistic Regression	25
2.11 Model Random Forest.....	26
2.12 Model CatBoost	27
2.13 Hyperparameter Model Awal	29
2.14 Hyperparameter Tuning	29
2.14.1 Tuning CatBoost	29
2.14.2 Tuning Logistic Regression	30
2.14.3 Tuning Random Forest.....	30
2.15 Metode Evaluasi.....	31
2.15.1 Confusion matrix	31
2.15.2 Accuracy	32
2.15.3 Precision.....	32
2.15.4 Recall	32
2.15.5 F1-score.....	32

2.15.6 F1 Macro	32
2.15.7 Feature importance.....	33
2.16 Kriteria Penentuan Model Terbaik	33
BAB III HASIL DAN PEMBAHASAN	34
3.1 Implementasi Penelitian.....	34
3.2 Hasil Pemuatan Dataset	34
3.3 Hasil Feature Engineering.....	36
3.3.1 Pemilihan fitur	36
3.3.2 Transformasi usia	37
3.3.3 Transformasi pengalaman coding	38
3.3.4 Pembentukan target.....	38
3.4 Hasil Analisis Eksploratif	39
3.5 Hasil Skenario A: Prediksi AISelect	40
3.5.1 Pembagian dan balancing data.....	40
3.5.2 Hasil evaluasi model AISelect	41
3.5.3 Classification report AISelect	41
3.5.4 Confusion matrix AISelect.....	42
3.5.5 Feature importance AISelect.....	43
3.6 Hasil Skenario B: Prediksi AIAgents Awal.....	44
3.6.1 Pembagian dan balancing data.....	44
3.6.2 Hasil evaluasi model AIAgents awal	45
3.6.3 Classification report AIAgents.....	45
3.6.4 Confusion matrix AIAgents awal.....	46
3.6.5 Feature importance AIAgents	47
3.7 Optimasi Model AIAgents	48
3.7.1 Hasil hyperparameter tuning CatBoost.....	48
3.7.2 Hasil random under-sampling.....	49

3.7.3 Hasil tuning Logistic Regression	51
3.7.4 Hasil tuning Random Forest	51
3.8 Perbandingan Hasil Optimasi	53
3.9 Feature Importance Random Forest Tuned.....	53
3.10 Perbandingan Seluruh Skenario	55
3.11 Pembahasan Pemilihan Fitur.....	56
3.12 Pembahasan Umum	57
BAB IV KESIMPULAN DAN SARAN	58
4.1 Kesimpulan	58
4.2 Saran	60
DAFTAR PUSTAKA	62

DAFTAR GAMBAR

Gambar 1. Contoh lima data asli Stack Overflow Developer Survey 2025	15
Gambar 2. Alur penelitian prediksi adopsi AI pada developer.....	19
Gambar 3. Arsitektur Logistic Regression	26
Gambar 4. Arsitektur Random Forest.....	27
Gambar 5. Arsitektur CatBoost	28
Gambar 6. Hasil pemuatan dan tiga contoh data asli	35
Gambar 7. Distribusi sentimen developer terhadap AI	40
Gambar 8. Confusion matrix model pada target AISelect.....	42
Gambar 9. Feature importance CatBoost pada target AISelect.....	43
Gambar 10. Confusion matrix awal pada target AIAgents.....	46
Gambar 11. Feature importance CatBoost pada target AIAgents.....	47
Gambar 12. Confusion matrix model setelah under-sampling	50
Gambar 13. Top 15 feature importance Random Forest tuned.....	54

DAFTAR TABEL

Tabel 1. Perangkat dan library penelitian	13
Tabel 2. Jumlah data pada setiap tahap	15
Tabel 3. Contoh data asli sebelum preprocessing.....	16
Tabel 4. Target klasifikasi.....	18
Tabel 5. Skenario pemodelan	25
Tabel 6. Hyperparameter model awal.....	29
Tabel 7. Ruang pencarian CatBoost	30
Tabel 8. Ruang pencarian Logistic Regression	30
Tabel 9. Ruang pencarian Random Forest.....	31
Tabel 10. Struktur confusion matrix.....	31
Tabel 11. Hasil evaluasi model pada target AISelect	41
Tabel 12. Confusion matrix AISelect	42
Tabel 13. Hasil evaluasi awal model pada target AIAgents	45
Tabel 14. Confusion matrix AIAgents awal	46
Tabel 15. Hasil model setelah under-sampling.....	50
Tabel 16. Confusion matrix setelah under-sampling.....	50
Tabel 17. Perbandingan model hasil optimasi AIAgents	53
Tabel 18. Perbandingan seluruh hasil penelitian	55
Tabel 19. Model terbaik setiap skenario.....	56

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan Artificial Intelligence atau AI telah memberikan perubahan yang cukup besar dalam berbagai bidang, termasuk bidang pengembangan perangkat lunak. AI tidak hanya digunakan untuk melakukan analisis data, tetapi juga mulai dimanfaatkan oleh developer dalam membantu proses penulisan kode, pencarian solusi, debugging, pembuatan dokumentasi, pengujian aplikasi, serta penyelesaian berbagai permasalahan teknis. Kehadiran AI Tools dan AI Agents membuat aktivitas pengembangan perangkat lunak dapat dilakukan dengan lebih cepat dan efisien [1].

Walaupun penggunaan AI dalam pengembangan perangkat lunak terus meningkat, tingkat adopsinya belum sama pada setiap developer. Sebagian developer telah menggunakan AI sebagai bagian dari aktivitas kerja sehari-hari, sementara sebagian lainnya masih jarang menggunakan atau bahkan belum menggunakannya. Perbedaan tersebut dapat dipengaruhi oleh berbagai karakteristik, seperti usia, pengalaman coding, pengalaman kerja, jenis pekerjaan, teknologi yang digunakan, lingkungan pengembangan, dan pandangan developer terhadap AI [2].

Perbedaan tingkat adopsi tersebut menghasilkan suatu permasalahan yang menarik untuk dianalisis. Data mengenai karakteristik developer dapat digunakan untuk melihat pola yang membedakan developer pengguna dan non-pengguna AI. Namun, data survei developer biasanya memiliki jumlah responden dan variabel yang besar, serta terdiri atas data numerik, kategorikal, jawaban rentang, jawaban kosong, dan jawaban dengan beberapa pilihan dalam satu kolom. Kondisi tersebut membuat analisis secara manual menjadi kurang efektif sehingga diperlukan pendekatan komputasional yang mampu mempelajari pola dari data secara otomatis.

Machine Learning merupakan bagian dari Artificial Intelligence yang memungkinkan komputer mempelajari pola berdasarkan data historis dan menghasilkan prediksi terhadap data baru. Salah satu pendekatan yang dapat digunakan adalah supervised learning dengan metode klasifikasi. Dalam klasifikasi, model dilatih menggunakan data yang telah memiliki kelas atau label untuk memprediksi kelas dari data yang belum pernah dipelajari sebelumnya. Pendekatan ini dapat digunakan untuk memprediksi apakah seorang developer termasuk pengguna atau non-pengguna teknologi AI berdasarkan karakteristik yang dimilikinya [3].

Penelitian ini menggunakan dataset **Stack Overflow Developer Survey 2025** yang berasal dari hasil survei terhadap developer dari berbagai negara. Dataset awal terdiri atas **49.191 responden dan 172 kolom** yang mencakup informasi demografi, pengalaman, jenis pekerjaan, teknologi yang digunakan, lingkungan pengembangan, kompensasi, serta penggunaan dan sentimen terhadap AI. Dari keseluruhan kolom tersebut, penelitian memilih 11 fitur prediktor yang dinilai relevan, yaitu Age, YearsCode, WorkExp, CompTotal, DevType, OpSys, DevEnvs, Language, Webframe, Database, dan AISent.

Pemilihan 11 fitur tersebut dilakukan dengan mempertimbangkan keterkaitan masing-masing fitur dengan adopsi AI, ketersediaan data, tipe data, tingkat kelengkapan data, serta hasil eksplorasi dan beberapa percobaan pemodelan. Fitur Age, YearsCode, dan WorkExp digunakan untuk menggambarkan umur serta tingkat pengalaman developer. Fitur CompTotal digunakan sebagai salah satu indikator kondisi profesional dan pekerjaan. Sementara itu, DevType, OpSys, DevEnvs, Language, Webframe, dan Database digunakan untuk menggambarkan ekosistem teknologi yang digunakan oleh developer. Fitur AISent digunakan untuk merepresentasikan pandangan atau sentimen developer terhadap AI.

Pemilihan fitur juga dilakukan dengan menghindari penggunaan atribut yang secara langsung menggambarkan nilai target. Hal ini dilakukan untuk mencegah terjadinya **data leakage**, yaitu kondisi ketika informasi dari target ikut digunakan dalam proses pelatihan sehingga hasil evaluasi model terlihat sangat tinggi tetapi tidak mencerminkan kemampuan prediksi yang sebenarnya. Oleh karena itu, fitur yang digunakan dibatasi pada karakteristik developer dan informasi pendukung yang secara logis dapat berhubungan dengan kecenderungan penggunaan AI [4].

Penelitian menggunakan dua target klasifikasi, yaitu AISelect dan AIAgents. Target AISelect digunakan untuk memprediksi apakah developer menggunakan AI Tools secara umum. Sementara itu, target AIAgents digunakan untuk memprediksi apakah developer telah menggunakan AI Agent dalam aktivitas pengembangan perangkat lunak. Kedua target diubah menjadi klasifikasi biner, yaitu jawaban yang diawali dengan “Yes” menjadi kelas 1, sedangkan jawaban yang diawali dengan “No” menjadi kelas 0.

Target AISelect menggambarkan bentuk penggunaan AI yang lebih umum, seperti penggunaan AI untuk membantu coding, pencarian informasi, dan penyelesaian masalah. Sementara itu, AIAgents menggambarkan bentuk adopsi AI yang lebih spesifik dan relatif belum digunakan oleh seluruh developer. Perbedaan karakteristik kedua target tersebut menyebabkan tingkat kesulitan prediksi yang juga berbeda. Prediksi penggunaan AI Agent diperkirakan lebih sulit

karena jumlah pengguna lebih sedikit dan pola penggunaannya lebih khusus dibandingkan penggunaan AI Tools secara umum.

Beberapa metode Machine Learning dapat digunakan untuk melakukan klasifikasi terhadap dataset tersebut, seperti Support Vector Machine, Decision Tree, Naive Bayes, Logistic Regression, Random Forest, gradient boosting, dan neural network. Dalam penelitian ini, model yang digunakan adalah **Logistic Regression, Random Forest, dan CatBoost**. Ketiga model dipilih karena memiliki karakteristik yang berbeda sehingga dapat memberikan perbandingan yang lebih lengkap.

Logistic Regression digunakan sebagai model baseline karena memiliki proses pelatihan yang relatif cepat, sederhana, serta mudah digunakan untuk klasifikasi biner. Model ini mempelajari hubungan antara fitur dan probabilitas suatu data termasuk ke dalam kelas tertentu. Namun, Logistic Regression memiliki keterbatasan dalam mempelajari hubungan non-linear dan interaksi fitur yang kompleks [5].

Random Forest merupakan metode ensemble yang menggabungkan beberapa decision tree untuk menghasilkan keputusan klasifikasi. Penggunaan banyak pohon keputusan dapat mengurangi risiko overfitting yang biasanya terjadi pada satu decision tree. Random Forest juga mampu menangani hubungan non-linear dan interaksi antarfitur, serta menyediakan feature importance untuk menunjukkan fitur yang banyak digunakan dalam proses pengambilan keputusan model [6].

CatBoost merupakan algoritma gradient boosting yang dirancang untuk menangani data tabular dan fitur kategorikal. Model ini sesuai digunakan pada dataset survei developer karena sebagian besar fitur yang digunakan berbentuk kategori. CatBoost juga memiliki mekanisme pengolahan fitur kategorikal yang dapat mengurangi kebutuhan encoding secara manual serta mampu mempelajari hubungan yang kompleks antarvariabel [7].

Selain karakteristik fitur, salah satu permasalahan dalam penelitian ini adalah distribusi kelas yang tidak seimbang. Pada target tertentu, jumlah data pengguna dan non-pengguna AI tidak memiliki proporsi yang sama. Apabila evaluasi hanya dilakukan menggunakan accuracy, model dapat terlihat memiliki performa yang baik walaupun lebih sering memprediksi kelas mayoritas. Oleh karena itu, penelitian menggunakan precision, recall, F1-score setiap kelas, F1 Macro, accuracy, dan confusion matrix dalam mengevaluasi hasil model [8].

F1 Macro digunakan sebagai metrik utama karena memberikan bobot yang sama terhadap setiap kelas. Dengan demikian, model tidak hanya dinilai berdasarkan kemampuannya mengenali kelas mayoritas, tetapi juga berdasarkan kemampuannya mengenali kelas minoritas. Penelitian juga menerapkan beberapa teknik penanganan ketidakseimbangan kelas, seperti

SMOTENC, class weight, dan random under-sampling. Proses balancing hanya diterapkan pada data training, sedangkan data testing tetap menggunakan distribusi aslinya agar hasil evaluasi tetap menggambarkan kondisi data sebenarnya.

Ide utama penelitian ini adalah membangun dan membandingkan beberapa model klasifikasi untuk mengetahui model yang paling baik dan konsisten dalam memprediksi adopsi AI pada developer. Penelitian tidak hanya membandingkan nilai accuracy, tetapi juga memperhatikan keseimbangan precision, recall, dan F1-score pada kedua kelas. Selain itu, feature importance digunakan untuk mengidentifikasi fitur yang menjadi sinyal prediktif utama dalam membedakan developer pengguna dan non-pengguna AI.

Hasil penelitian diharapkan dapat memberikan gambaran mengenai pola adopsi AI di kalangan developer. Informasi mengenai karakteristik developer yang berkaitan dengan penggunaan AI dapat memberikan pemahaman yang lebih terukur mengenai perkembangan AI dalam industri perangkat lunak. Akan tetapi, hasil feature importance dalam penelitian ini hanya menunjukkan hubungan prediktif berdasarkan pola data dan tidak dapat digunakan untuk menyimpulkan hubungan sebab-akibat.

Berdasarkan uraian tersebut, penelitian ini mengangkat judul “Prediksi Adopsi Artificial Intelligence pada Developer Menggunakan Logistic Regression, Random Forest, dan CatBoost Berdasarkan Stack Overflow Developer Survey 2025.”

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah dijelaskan, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana melakukan pemilihan fitur, pembersihan data, dan preprocessing pada dataset Stack Overflow Developer Survey 2025 agar dapat digunakan untuk memprediksi adopsi AI pada developer?
2. Bagaimana penerapan Logistic Regression, Random Forest, dan CatBoost dalam memprediksi penggunaan AI Tools berdasarkan target AISelect?
3. Bagaimana penerapan Logistic Regression, Random Forest, dan CatBoost dalam memprediksi penggunaan AI Agent berdasarkan target AIAgents?
4. Bagaimana pengaruh penanganan ketidakseimbangan kelas dan hyperparameter tuning terhadap performa model dalam memprediksi adopsi AI?
5. Model manakah yang menghasilkan performa terbaik berdasarkan F1 Macro serta fitur apa saja yang menjadi sinyal prediktif utama dalam memprediksi adopsi AI pada developer?

1.3 Batasan Masalah

Agar penelitian lebih terarah dan tidak menyimpang dari tujuan yang telah ditentukan, batasan masalah dalam penelitian ini adalah sebagai berikut:

1. Dataset yang digunakan hanya berasal dari **Stack Overflow Developer Survey 2025**, dengan data awal sebanyak 49.191 responden dan 172 kolom.
2. Penelitian menggunakan 11 fitur prediktor, yaitu Age, YearsCode, WorkExp, CompTotal, DevType, OpSys, DevEnvs, Language, Webframe, Database, dan AISent.
3. Target prediksi dibatasi pada AISelect untuk penggunaan AI Tools secara umum dan AIAgents untuk penggunaan AI Agent, dengan kelas biner 1 untuk jawaban “Yes” dan 0 untuk jawaban “No”.
4. Metode klasifikasi yang dibandingkan dibatasi pada Logistic Regression, Random Forest, dan CatBoost, dengan penanganan ketidakseimbangan kelas menggunakan SMOTENC, class weight, dan random under-sampling pada data training.
5. Evaluasi model dibatasi menggunakan precision, recall, F1-score setiap kelas, F1 Macro, accuracy, confusion matrix, dan feature importance. Penelitian berfokus pada hubungan prediktif dan tidak membahas hubungan sebab-akibat antarvariabel.

1.4 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah ditentukan, tujuan penelitian ini adalah sebagai berikut:

1. Melakukan pemilihan fitur, pembersihan data, dan preprocessing terhadap dataset Stack Overflow Developer Survey 2025 agar dapat digunakan dalam proses pemodelan Machine Learning.
2. Menerapkan dan mengevaluasi Logistic Regression, Random Forest, dan CatBoost untuk memprediksi penggunaan AI Tools berdasarkan target AISelect.
3. Menerapkan dan mengevaluasi Logistic Regression, Random Forest, dan CatBoost untuk memprediksi penggunaan AI Agent berdasarkan target AIAgents.
4. Menganalisis pengaruh penanganan ketidakseimbangan kelas dan hyperparameter tuning terhadap peningkatan performa model prediksi adopsi AI.
5. Menentukan model dengan performa terbaik berdasarkan F1 Macro serta mengidentifikasi fitur yang menjadi sinyal prediktif utama terhadap adopsi AI pada developer.

BAB II

METODOLOGI PENELITIAN

2.1 Desain Penelitian

Penelitian ini menggunakan pendekatan kuantitatif eksperimental dengan menerapkan metode Machine Learning untuk melakukan klasifikasi terhadap adopsi Artificial Intelligence pada developer. Pendekatan yang digunakan adalah **supervised learning**, yaitu model mempelajari hubungan antara sejumlah fitur prediktor dan target yang telah memiliki label kelas.

Tugas klasifikasi dalam penelitian ini terdiri atas dua skenario. Skenario pertama menggunakan target AISelect untuk memprediksi apakah developer menggunakan AI Tools secara umum. Skenario kedua menggunakan target AIAgents untuk memprediksi apakah developer menggunakan AI Agent dalam aktivitas pekerjaannya. Kedua target tersebut dibentuk sebagai klasifikasi biner dengan kelas 0 yang menunjukkan jawaban “No” dan kelas 1 yang menunjukkan jawaban “Yes”.

Penelitian membandingkan tiga algoritma klasifikasi, yaitu Logistic Regression, Random Forest, dan CatBoost. Logistic Regression digunakan sebagai model baseline linear, Random Forest digunakan sebagai model ensemble berbasis decision tree, sedangkan CatBoost digunakan sebagai model gradient boosting yang mampu mengolah data tabular dan kategorikal. Perbandingan ketiga model dilakukan untuk memperoleh model yang memiliki performa paling seimbang dalam mengenali pengguna maupun non-pengguna AI.

Selain pemodelan awal, penelitian juga melakukan optimasi pada target AIAgents. Optimasi tersebut dilakukan karena distribusi kelas pada target AIAgents lebih tidak seimbang dan pengguna AI Agent lebih sulit dikenali dibandingkan pengguna AI Tools secara umum. Optimasi dilakukan menggunakan class weighting, random under-sampling, dan hyperparameter tuning dengan GridSearchCV.

2.2 Perangkat dan Lingkungan Penelitian

Implementasi penelitian dilakukan menggunakan bahasa pemrograman Python melalui Google Colab. Penggunaan Google Colab memudahkan proses pengolahan dataset, pelatihan model, pembuatan visualisasi, dan penyimpanan notebook pada Google Drive.

Perangkat lunak dan library yang digunakan dalam penelitian ditunjukkan pada Tabel 2.1.

Tabel 1. Perangkat dan library penelitian

No.	Perangkat atau library	Fungsi
1	Google Colab	Lingkungan untuk menjalankan notebook Python

2	Google Drive	Penyimpanan dataset dan file notebook
3	Python	Bahasa pemrograman utama
4	Pandas	Membaca dan mengolah data tabular
5	NumPy	Melakukan operasi numerik
6	Matplotlib	Membuat grafik dan visualisasi
7	Seaborn	Membuat visualisasi distribusi dan confusion matrix
8	Scikit-learn	Preprocessing, pembagian data, pemodelan, dan evaluasi
9	Imbalanced-learn	Implementasi SMOTENC dan RandomUnderSampler
10	CatBoost	Implementasi model CatBoostClassifier

Untuk menjaga hasil eksperimen agar dapat direproduksi, penelitian menetapkan nilai `random_state = 42` pada proses pembagian data, balancing, dan pelatihan model yang mendukung parameter tersebut.

2.3 Dataset Penelitian

2.3.1 Sumber dan karakteristik dataset

Dataset yang digunakan adalah **Stack Overflow Developer Survey 2025**. Dataset ini merupakan data hasil survei terhadap developer yang memuat informasi mengenai karakteristik responden, pengalaman coding, pengalaman kerja, pekerjaan, kompensasi, teknologi yang digunakan, serta sikap dan penggunaan AI [9].

Dataset disimpan dalam file bernama `results.csv`. Dataset awal memiliki **49.191 baris responden dan 172 kolom**. Penelitian tidak menggunakan seluruh kolom tersebut karena tidak semua variabel berhubungan dengan tujuan prediksi. Pemilihan kolom dilakukan berdasarkan relevansi konseptual terhadap adopsi AI, kelengkapan data, tipe data, serta hasil eksplorasi dan percobaan pemodelan.

Notebook hanya membaca 13 kolom yang diperlukan, terdiri atas 11 fitur prediktor dan dua target klasifikasi. Pemilihan kolom secara langsung ketika proses pembacaan data juga dilakukan agar penggunaan memori dan waktu pemrosesan menjadi lebih efisien.

2.3.2 Jumlah data pada setiap skenario

Setelah target diubah menjadi kelas biner, data yang tidak memiliki jawaban “Yes” atau “No” pada target tidak digunakan dalam skenario terkait. Oleh karena itu, jumlah data valid pada `AISelect` dan `AIAgents` berbeda.

Tabel 2. Jumlah data pada setiap tahap

Tahap	AISelect	AIAgents
Data awal keseluruhan	49.191	49.191
Data dengan target valid	33.720	31.919
Data training sebelum balancing	26.976	25.535
Data testing	6.744	6.384
Data training setelah SMOTENC	42.350	35.270
Data training setelah under-sampling	—	15.800

Data untuk target AISelect yang dapat digunakan berjumlah 33.720 data. Sebanyak 26.976 data digunakan sebagai data training dan 6.744 data digunakan sebagai data testing. Setelah dilakukan SMOTENC, jumlah data training menjadi 42.350 data.

Data untuk target AIAgents yang dapat digunakan berjumlah 31.919 data. Sebanyak 25.535 data digunakan sebagai data training dan 6.384 data digunakan sebagai data testing. Setelah dilakukan SMOTENC, data training meningkat menjadi 35.270 data. Pada skenario random under-sampling, data training dikurangi dan diseimbangkan menjadi 15.800 data.

2.4 Contoh Data Asli

Data asli masih berbentuk jawaban survei. Beberapa kolom berisi rentang nilai, teks panjang, nilai kosong, dan beberapa pilihan yang dipisahkan dengan tanda titik koma. Contohnya, satu responden dapat menggunakan beberapa bahasa pemrograman, database, sistem operasi, dan lingkungan pengembangan sekaligus.

Notebook yang digunakan saat ini menampilkan tiga data asli melalui perintah berikut:

```
display(df_raw.head(5))
```

Gambar 1. Contoh lima data asli Stack Overflow Developer Survey 2025

Tiga data asli yang telah tersimpan dalam output notebook ditunjukkan secara ringkas pada Tabel 3.

Tabel 3. Contoh data asli sebelum preprocessing

No.	Age	YearsCode	WorkExp	DevType	AISelect	AI Sent	AI Agents
1	25– 34 years old	14	8	Developer, mobile	Yes, monthly or infrequently	Indifferent	Yes, monthly or infrequently
2	25– 34 years old	10	2	Developer, back-end	Yes, weekly	Indifferent	No, and I don’t plan to
3	35– 44 years old	12	10	Developer, front-end	Yes, daily	Favorable	Yes, weekly

Tabel tersebut hanya digunakan sebagai gambaran ringkas. Untuk laporan final, Gambar 2.1 tetap harus berisi **lima baris output asli** dari `df_raw.head(5)` agar sesuai dengan ketentuan soal.

2.5 Variabel Penelitian

2.5.1 Fitur prediktor

Penelitian menggunakan 11 fitur prediktor yang terdiri atas empat fitur numerik dan tujuh fitur kategorikal.

No.	Nama fitur	Kolom asli	Tipe	Keterangan
1	Age	Age	Numerik setelah transformasi	Rentang usia developer
2	YearsCode	YearsCode	Numerik	Lama pengalaman melakukan coding
3	WorkExp	WorkExp	Numerik	Lama pengalaman kerja profesional

4	CompTotal	CompTotal	Numerik	Total kompensasi yang dilaporkan responden
5	DevType	DevType	Kategorikal	Jenis pekerjaan atau peran developer
6	OpSys	OpSysProfessional use	Kategorikal	Sistem operasi yang digunakan secara profesional
7	DevEnvs	DevEnvsHaveWorkedWith	Kategorikal	Lingkungan pengembangan yang pernah digunakan
8	Language	LanguageHaveWorkedWith	Kategorikal	Bahasa pemrograman yang pernah digunakan
9	Webframe	WebframeHaveWorkedWith	Kategorikal	Web framework yang pernah digunakan
10	Database	DatabaseHaveWorkedWith	Kategorikal	Teknologi database yang pernah digunakan
11	AISent	AISent	Kategorikal	Sentimen atau pandangan developer terhadap AI

Fitur Age, YearsCode, dan WorkExp dipilih untuk menggambarkan tingkat usia dan pengalaman responden. Pengalaman developer diperkirakan dapat menjadi sinyal yang membedakan pola penggunaan teknologi AI.

Fitur CompTotal digunakan sebagai informasi mengenai kondisi profesional responden. Namun, nilai kompensasi pada dataset berasal dari negara dan mata uang yang berbeda. Oleh karena itu, fitur ini hanya digunakan sebagai sinyal prediktif dan tidak diartikan sebagai perbandingan nilai pendapatan langsung antarnegara.

Fitur DevType, OpSys, DevEnvs, Language, Webframe, dan Database digunakan untuk menggambarkan lingkungan teknologi developer. Developer dengan pekerjaan dan ekosistem teknologi berbeda dapat memiliki kebutuhan dan kecenderungan penggunaan AI yang berbeda.

Fitur AISent digunakan untuk menggambarkan sentimen developer terhadap AI. Sikap yang favorable, unfavorable, atau indifferent dapat membantu model mengenali pola adopsi AI. Namun, kontribusi prediktif fitur tersebut tidak dapat langsung diartikan sebagai hubungan sebab-akibat.

2.5.2 Target klasifikasi

Penelitian menggunakan dua target klasifikasi sebagaimana ditunjukkan pada Tabel 2.5.

Tabel 4. Target klasifikasi

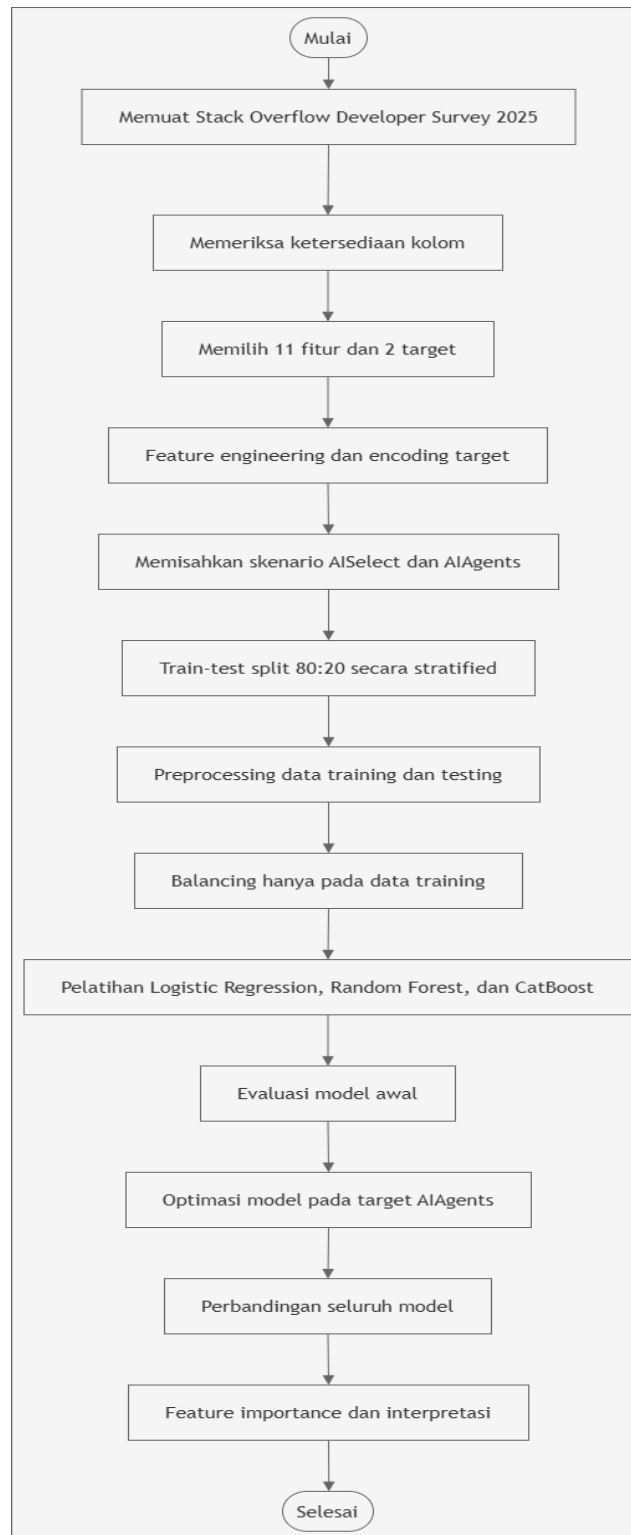
Target	Kelas 0	Kelas 1	Fungsi
AISelect	Jawaban diawali “No”	Jawaban diawali “Yes”	Memprediksi penggunaan AI Tools secara umum
AIAgents	Jawaban diawali “No”	Jawaban diawali “Yes”	Memprediksi penggunaan AI Agent dalam pekerjaan

Target AISelect digunakan untuk melihat penggunaan alat AI secara umum. Target ini mencakup penggunaan AI dengan frekuensi harian, mingguan, bulanan, maupun tidak rutin selama jawaban responden diawali dengan “Yes”.

Target AIAgents lebih khusus karena mengukur penggunaan AI Agent dalam lingkungan kerja. Target ini memiliki distribusi kelas yang lebih tidak seimbang sehingga memerlukan perhatian khusus terhadap kemampuan model dalam mengenali kelas pengguna AI Agent.

2.6 Alur Penelitian

Tahapan penelitian dimulai dari pembacaan dataset hingga interpretasi hasil model. Alur penelitian disusun agar seluruh proses pengolahan data dilakukan secara sistematis dan menghindari kebocoran informasi dari data testing ke data training.



Gambar 2. Alur penelitian prediksi adopsi AI pada developer

Isi diagram alur Gambar 2:

Tahapan penelitian dijelaskan sebagai berikut:

1. **Memuat dataset**

Dataset results.csv dimuat dari Google Drive. Sistem memeriksa keberadaan file dan kolom yang dibutuhkan.

2. **Memilih variable**

Sebanyak 11 fitur prediktor dan dua target dipilih dari 172 kolom awal.

3. **Melakukan feature engineering**

Rentang umur diubah menjadi nilai numerik, jawaban khusus pada YearsCode dikonversi, nama kolom diringkaskan, dan target diubah menjadi kelas biner.

4. **Membentuk dua scenario**

Data dipisahkan berdasarkan target valid untuk skenario AISelect dan AIAgents.

5. **Membagi data**

Data dibagi menjadi 80% data training dan 20% data testing dengan stratifikasi kelas.

6. **Melakukan preprocessing**

Tahap ini meliputi penanganan missing value, pemadatan kategori, clipping nilai ekstrem, transformasi kompensasi, encoding, dan scaling.

7. **Menangani class imbalance**

SMOTENC digunakan pada pemodelan awal. Class weight dan random under-sampling digunakan pada proses optimasi AIAgents.

8. **Melatih model**

Logistic Regression, Random Forest, dan CatBoost dilatih menggunakan data training.

9. **Mengevaluasi model**

Model dievaluasi menggunakan precision, recall, F1-score, F1 Macro, accuracy, dan confusion matrix.

10. **Melakukan tuning**

GridSearchCV digunakan untuk mencari kombinasi hyperparameter terbaik berdasarkan F1 Macro.

11. **Menginterpretasikan model**

Feature importance digunakan untuk melihat fitur yang paling informatif bagi model.

2.7 Preprocessing Data

2.7.1 Pemeriksaan kolom

Notebook terlebih dahulu membaca nama seluruh kolom dalam dataset. Daftar tersebut dibandingkan dengan kolom wajib yang telah ditentukan. Apabila terdapat kolom wajib yang tidak ditemukan, proses dihentikan dan menampilkan pesan kesalahan.

Pemeriksaan ini dilakukan untuk mencegah proses pemodelan berjalan menggunakan struktur dataset yang tidak sesuai.

2.7.2 Pemilihan dan penggantian nama kolom

Kolom dengan nama panjang diganti menjadi nama yang lebih ringkas. Contohnya:

- OpSysProfessional use menjadi OpSys;
- DevEnvsHaveWorkedWith menjadi DevEnvs;
- LanguageHaveWorkedWith menjadi Language;
- WebframeHaveWorkedWith menjadi Webframe;
- DatabaseHaveWorkedWith menjadi Database.

Penggantian nama tersebut tidak mengubah isi data dan hanya dilakukan agar source code lebih mudah dibaca.

2.7.3 Transformasi usia

Kolom Age pada data asli berbentuk rentang usia. Nilai tersebut diubah menjadi angka tengah dari setiap rentang. Contohnya, rentang 25–34 years old diubah menjadi:

$$Age = \frac{25 + 34}{2} = 29,5$$

Rentang 35–44 years old diubah menjadi 39,5. Jawaban Under 18 years old diubah menjadi 17, sedangkan jawaban 65 years or older diubah menjadi 70.

Transformasi ini dilakukan agar informasi usia dapat diproses sebagai fitur numerik.

2.7.4 Transformasi pengalaman coding

Kolom YearsCode dikonversi menjadi nilai numerik. Jawaban khusus diproses dengan ketentuan berikut:

Jawaban asli	Nilai hasil transformasi
Less than 1 year	0,5
More than 50 years	51
Nilai tahun lainnya	Diubah langsung menjadi numerik

Kolom WorkExp juga dikonversi menjadi data numerik menggunakan fungsi `pd.to_numeric`.

2.7.5 Pembentukan target biner

Target ASelect dan AIAgents diubah menjadi bilangan biner. Jawaban yang diawali kata Yes diberikan nilai 1, sedangkan jawaban yang diawali kata No diberikan nilai 0.

Jawaban lain atau nilai kosong tidak diberi kelas sehingga tidak digunakan pada skenario target tersebut.

2.7.6 Pembagian data training dan testing

Data dibagi menjadi:

- 80% data training;
- 20% data testing.

Pembagian dilakukan menggunakan `train_test_split` dengan:

```
test_size      = 0,20  
stratify       = target  
random_state   = 42
```

Parameter stratify digunakan agar proporsi kelas pada data training dan testing tetap mengikuti distribusi target sebelum pembagian.

2.7.7 Pemadatan data multi-value

Beberapa kolom kategorikal dapat berisi beberapa jawaban dalam satu sel. Contohnya:

Python;JavaScript;SQL

Pada pemodelan awal, kategori multi-value dipadatkan menjadi satu kategori yang paling umum. Prosesnya adalah:

1. Memisahkan jawaban berdasarkan tanda titik koma.
2. Menghitung frekuensi kategori pada data training.
3. Memilih 25 kategori terpopuler pada setiap fitur.
4. Memilih kategori paling populer yang terdapat dalam jawaban responden.
5. Mengubah kategori yang tidak termasuk 25 kategori terpopuler menjadi Lainnya.
6. Mengubah nilai kosong menjadi Tidak diketahui.

Daftar kategori populer hanya dipelajari dari data training. Data testing mengikuti pemetaan yang dibentuk dari data training agar tidak terjadi data leakage.

2.7.8 Penanganan missing value

Missing value numerik diisi menggunakan nilai median dari data training. Strategi median dipilih karena lebih tahan terhadap nilai ekstrem dibandingkan nilai rata-rata.

Missing value kategorikal diisi dengan kategori:

Tidak diketahui

Nilai median dan pola imputasi dipelajari hanya dari data training, kemudian diterapkan pada data testing.

2.7.9 Penanganan nilai ekstrem

Nilai numerik dibatasi menggunakan persentil ke-0,5 dan persentil ke-99,5 dari data training. Nilai yang berada di bawah batas bawah diubah menjadi batas bawah, sedangkan nilai yang berada di atas batas atas diubah menjadi batas atas.

Proses clipping digunakan agar nilai ekstrem tidak terlalu mendominasi proses pelatihan model. Batas clipping dari data training juga diterapkan pada data testing.

2.7.10 Transformasi kompensasi

Fitur *CompTotal* memiliki rentang yang sangat besar. Selain itu, responden berasal dari negara dan mata uang berbeda. Setelah clipping, nilai kompensasi ditransformasi menggunakan fungsi:

$$CompTotal_{baru} = \log(1 + CompTotal)$$

Transformasi *log1p* membantu mengurangi kemencengan distribusi dan pengaruh nilai kompensasi yang sangat besar.

2.7.11 Encoding data kategorikal

Sebelum SMOTENC, fitur kategorikal diubah menjadi kode numerik menggunakan *OrdinalEncoder*. Kategori yang tidak dikenali pada data testing diberikan nilai -1.

Setelah proses SMOTENC:

- Logistic Regression dan Random Forest menggunakan *OneHotEncoder*;
- CatBoost menerima kategori hasil encoding yang dikonversi kembali menjadi tipe string.

Pada optimasi CatBoost, fitur kategorikal tidak melalui one-hot encoding. Missing value diisi dengan Tidak diketahui, kemudian fitur kategorikal diberikan secara langsung melalui parameter *cat_features*.

2.7.12 Standardisasi fitur

Standardisasi diterapkan pada fitur numerik dalam model Logistic Regression menggunakan *StandardScaler*. Standardisasi membantu menyetarakan skala fitur sehingga fitur dengan nilai besar tidak mendominasi proses pembentukan koefisien.

Random Forest tidak memerlukan standardisasi numerik karena keputusan model dibentuk berdasarkan pemisahan nilai pada decision tree.

2.8 Penanganan Ketidakseimbangan Kelas

2.8.1 SMOTENC

SMOTENC digunakan dalam pemodelan awal pada target AISelect dan AIAgents. SMOTENC merupakan pengembangan SMOTE yang dapat menangani kombinasi fitur numerik dan kategorikal. Metode ini membuat data sintetis pada kelas minoritas berdasarkan kedekatan antardata. [S13]

Parameter yang digunakan adalah:

`k_neighbors = 5`

`random_state = 42`

Nilai `k_neighbors` ditentukan secara adaptif berdasarkan jumlah data kelas minoritas. Dalam dataset penelitian, jumlah kelas minoritas mencukupi sehingga digunakan nilai maksimum lima tetangga.

SMOTENC hanya diterapkan pada data training. Data testing tidak diseimbangkan agar evaluasi tetap mencerminkan distribusi data sebenarnya.

2.8.2 Class weighting

Class weighting diterapkan pada optimasi CatBoost menggunakan:

<code>auto_class_weights = Balanced</code>
--

Metode ini memberikan bobot yang lebih besar pada kesalahan klasifikasi kelas minoritas. Dengan demikian, model terdorong untuk meningkatkan kemampuannya dalam mengenali pengguna AI Agent tanpa menghapus data kelas mayoritas.

2.8.3 Random under-sampling

Random under-sampling digunakan pada optimasi Logistic Regression dan Random Forest untuk target AIAgents. Metode ini mengurangi jumlah kelas mayoritas hingga sama dengan jumlah kelas minoritas.

Data training AIAgents yang awalnya berjumlah 25.535 data berubah menjadi 15.800 data dengan distribusi 50:50. Data testing tetap berjumlah 6.384 dan tidak mengalami perubahan. Kelebihan under-sampling adalah mempercepat pelatihan dan mengurangi dominasi kelas mayoritas. Namun, metode ini memiliki risiko menghilangkan sebagian informasi penting dari kelas mayoritas.

2.9 Skenario Pemodelan

Penelitian menggunakan tiga kelompok eksperimen.

Tabel 5. Skenario pemodelan

Skenario	Target	Balancing	Model
A	AISelect	SMOTENC	Logistic Regression, Random Forest, CatBoost
B	AIAgents	SMOTENC	Logistic Regression, Random Forest, CatBoost
C	AIAgents	Class weight dan under-sampling	Logistic Regression, Random Forest, CatBoost dengan tuning

Skenario A bertujuan memprediksi penggunaan AI Tools secara umum. Skenario B bertujuan memprediksi penggunaan AI Agent menggunakan konfigurasi awal. Skenario C dilakukan untuk meningkatkan kemampuan model dalam mengenali kelas pengguna AI Agent.

2.10 Model Logistic Regression

Logistic Regression merupakan metode klasifikasi yang menghitung probabilitas suatu data masuk ke dalam kelas tertentu. Model membentuk kombinasi linear dari seluruh fitur, kemudian mengubah hasilnya menjadi nilai probabilitas menggunakan fungsi sigmoid [10].

Fungsi sigmoid dirumuskan sebagai:

$$P(y = 1|x) = \frac{1}{1 + e^{-z}}$$

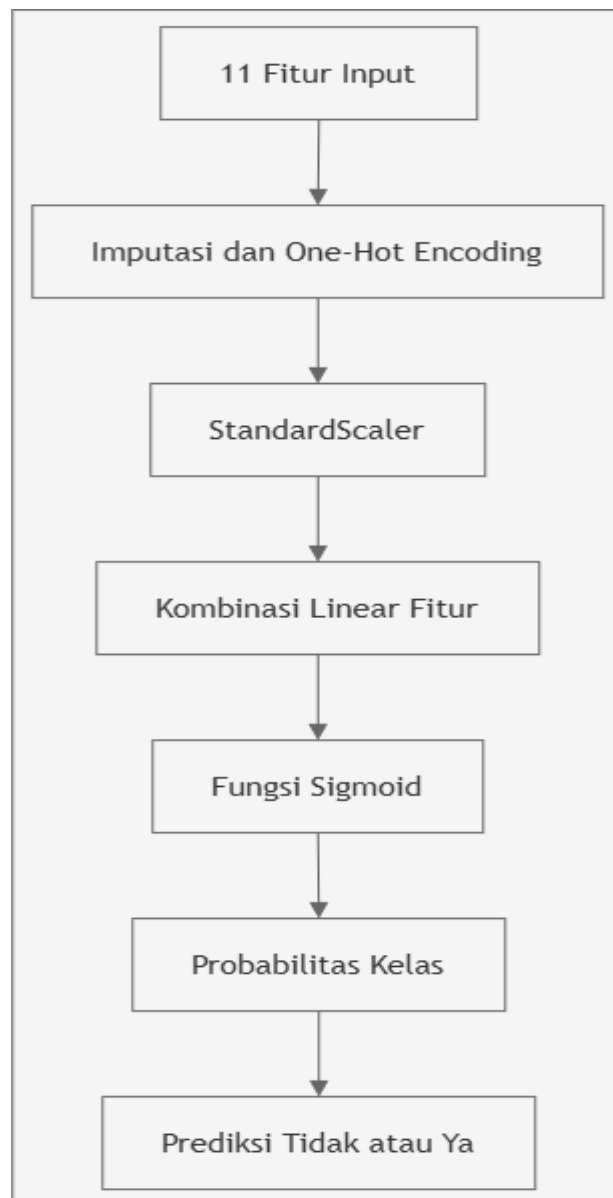
dengan:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

Keterangan:

- $P(y=1|x)$ adalah probabilitas data termasuk kelas 1;
- β_0 adalah intercept;
- $\beta_1, \dots, \beta_n$ adalah koefisien model;
- $x_1, \dots, x_n$ adalah fitur prediktor.

Apabila probabilitas melewati batas keputusan, data diklasifikasikan sebagai kelas 1. Sebaliknya, data diklasifikasikan sebagai kelas 0.



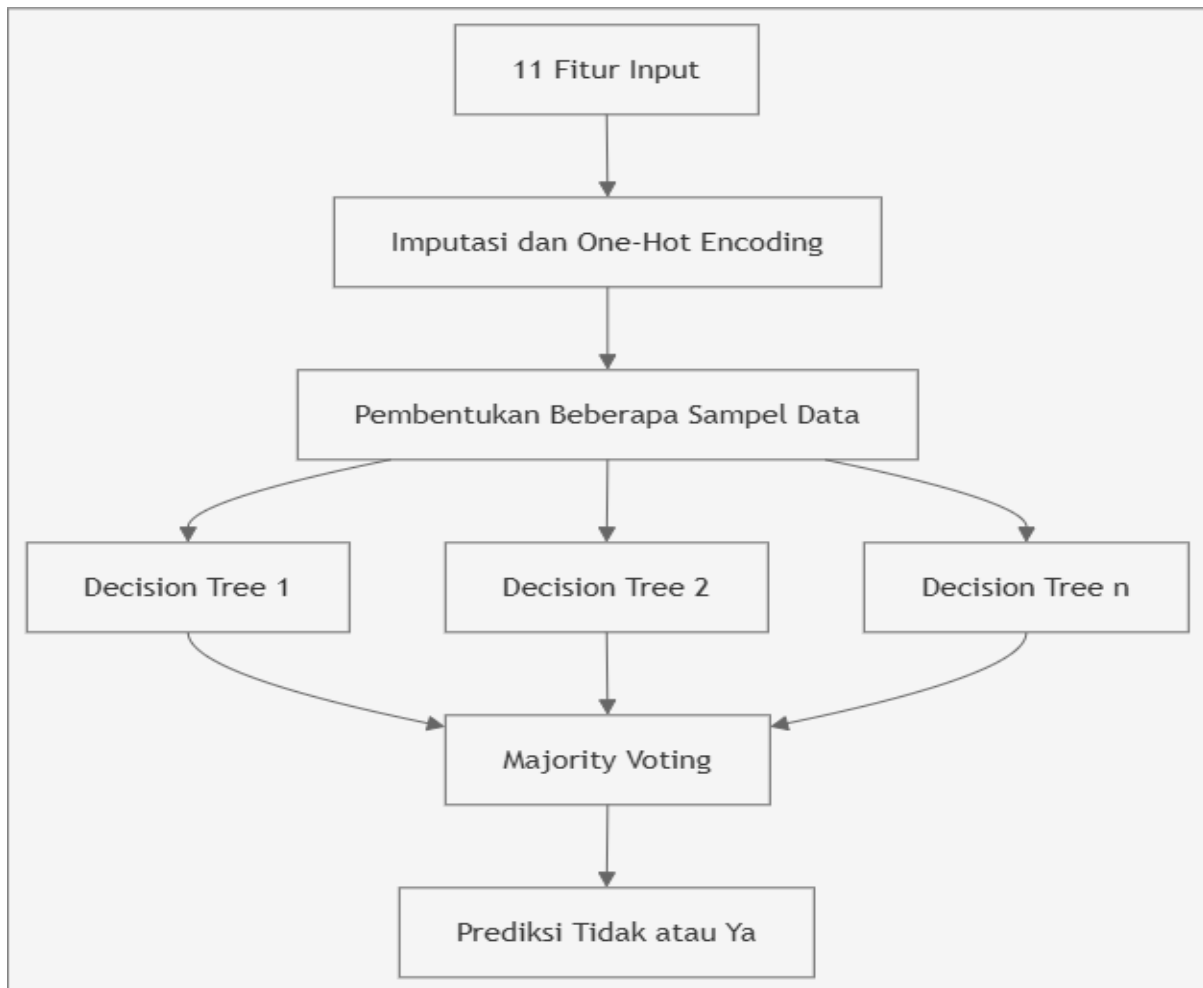
Gambar 3. Arsitektur Logistic Regression

Logistic Regression digunakan sebagai baseline karena sederhana, cepat, dan memberikan pembandingan terhadap model berbasis pohon.

2.11 Model Random Forest

Random Forest merupakan metode ensemble yang membangun banyak decision tree menggunakan sampel dan subset fitur yang berbeda. Prediksi akhir klasifikasi diperoleh melalui suara mayoritas dari seluruh pohon [2].

Penggunaan beberapa pohon membuat Random Forest lebih stabil dibandingkan satu decision tree. Model ini juga mampu mempelajari pola non-linear dan interaksi antarfitur.

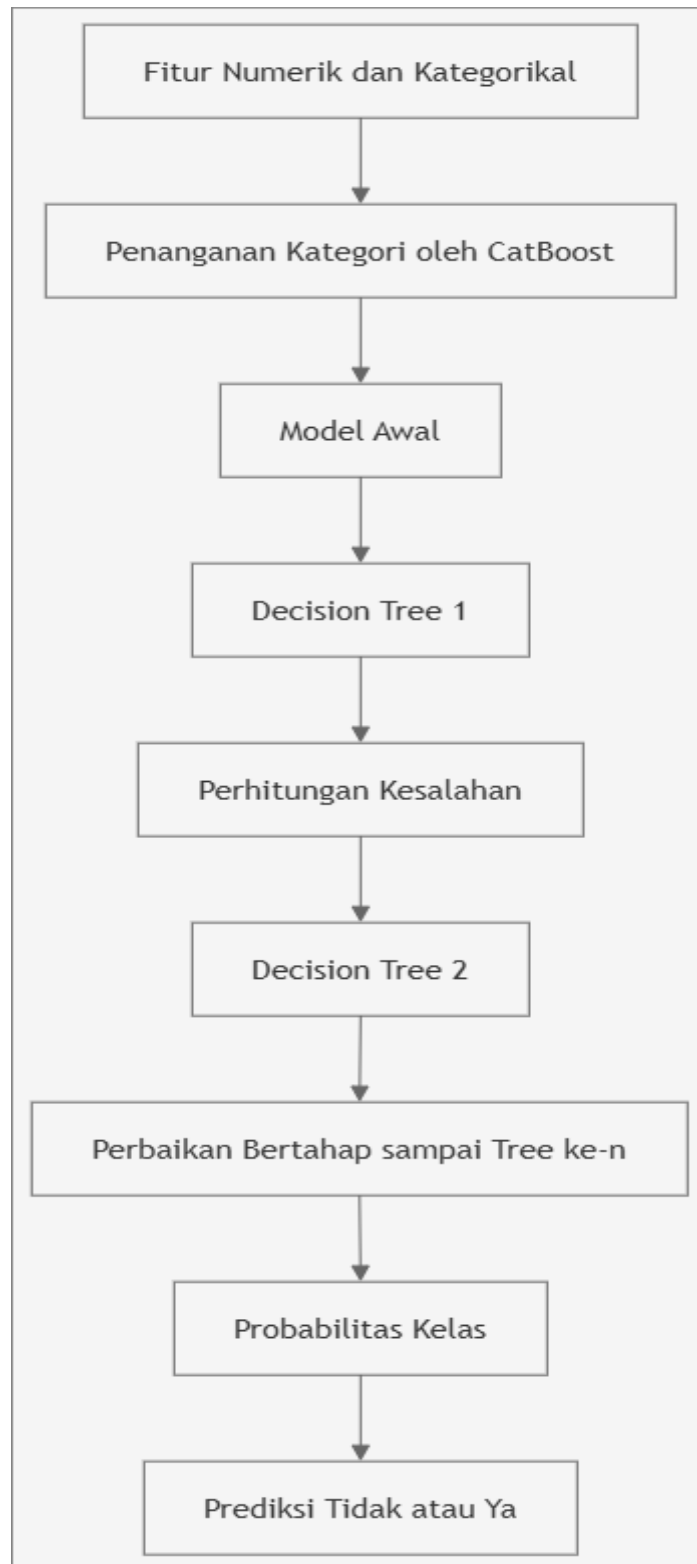


Gambar 4. Arsitektur Random Forest

Random Forest juga menyediakan feature importance berdasarkan seberapa besar suatu fitur membantu mengurangi ketidakmurnian pada proses pemisahan decision tree.

2.12 Model CatBoost

CatBoost merupakan algoritma gradient boosting berbasis decision tree yang dikembangkan untuk mengolah data tabular, khususnya data yang memiliki banyak fitur kategorikal. CatBoost membentuk pohon secara bertahap dengan memperbaiki kesalahan model sebelumnya [3]. Setiap pohon baru diarahkan untuk mempelajari residual atau kesalahan prediksi dari kumpulan pohon sebelumnya. Prediksi akhir merupakan gabungan kontribusi seluruh pohon.



Gambar 5. Arsitektur CatBoost

CatBoost dipilih karena dataset survei memiliki banyak fitur kategorikal, seperti pekerjaan, sistem operasi, bahasa pemrograman, framework, database, lingkungan pengembangan, dan sentimen terhadap AI.

2.13 Hyperparameter Model Awal

Konfigurasi model awal ditunjukkan pada Tabel 6.

Tabel 6. Hyperparameter model awal

Model	Hyperparameter	Nilai
Logistic Regression	max_iter	2.000
Logistic Regression	random_state	42
Random Forest	n_estimators	120
Random Forest	max_depth	18
Random Forest	min_samples_leaf	2
Random Forest	random_state	42
CatBoost	iterations	250
CatBoost	depth	6
CatBoost	learning_rate	0,10
CatBoost	loss_function	Logloss
CatBoost	eval_metric	F1
CatBoost	random_seed	42

Nilai max_iter = 2000 pada Logistic Regression digunakan untuk memberikan jumlah iterasi yang cukup hingga proses optimasi mencapai konvergensi.

Pada Random Forest, n_estimators = 120 menunjukkan bahwa model menggunakan 120 decision tree. Nilai max_depth = 18 membatasi kedalaman setiap pohon, sedangkan min_samples_leaf = 2 menentukan bahwa setiap daun minimal memiliki dua sampel.

Pada CatBoost, iterations = 250 menunjukkan jumlah tahapan boosting. Nilai depth = 6 menentukan kedalaman pohon, sedangkan learning_rate = 0,10 mengatur besarnya kontribusi setiap pohon baru.

2.14 Hyperparameter Tuning

Hyperparameter tuning dilakukan menggunakan GridSearchCV dengan tiga fold cross-validation. Metrik yang digunakan dalam pencarian parameter adalah f1_macro.

2.14.1 Tuning CatBoost

CatBoost menggunakan konfigurasi dasar:

iterations = 200

auto_class_weights = Balanced

random_state = 42
cross-validation = 3 fold
scoring = F1 Macro

Ruang pencarian parameter ditunjukkan pada Tabel 7.

Tabel 7. Ruang pencarian CatBoost

Hyperparameter	Nilai yang diuji
depth	4 dan 6
learning_rate	0,05 dan 0,10
l2_leaf_reg	1 dan 5

Jumlah kombinasi yang diuji adalah:

$$2 \times 2 \times 2 = 8 \text{ kombinasi}$$

Dengan tiga fold cross-validation, dilakukan 24 proses pelatihan. Konfigurasi terbaik yang diperoleh adalah:

depth = 6
learning_rate = 0,10
l2_leaf_reg = 1

2.14.2 Tuning Logistic Regression

Ruang pencarian Logistic Regression ditunjukkan pada Tabel 8.

Tabel 8. Ruang pencarian Logistic Regression

Hyperparameter	Nilai yang diuji
C	0,01; 0,1; 1,0; dan 10,0
penalty	L2
cross-validation	3 fold
scoring	F1 Macro

Empat kombinasi parameter diuji melalui 12 proses pelatihan. Parameter terbaik yang diperoleh adalah:

C = 0,01
penalty = L2

2.14.3 Tuning Random Forest

Ruang pencarian Random Forest ditunjukkan pada Tabel 9.

Tabel 9. Ruang pencarian Random Forest

Hyperparameter	Nilai yang diuji
n_estimators	100 dan 150
max_depth	10, 15, dan 20
min_samples_leaf	2 dan 5
cross-validation	3 fold
scoring	F1 Macro

Jumlah kombinasi parameter Random Forest adalah:

$$2 \times 3 \times 2 = 12 \text{ kombinasi}$$

Dengan tiga fold cross-validation, dilakukan 36 proses pelatihan. Parameter terbaik yang diperoleh adalah:

n_estimators = 150

max_depth = 20

min_samples_leaf = 2

2.15 Metode Evaluasi

Evaluasi model dilakukan menggunakan confusion matrix, accuracy, precision, recall, F1-score, dan F1 Macro. Penggunaan beberapa metrik diperlukan karena distribusi kelas pada dataset tidak seimbang. Accuracy saja dapat memberikan kesimpulan yang kurang tepat ketika model lebih sering memprediksi kelas mayoritas [5].

2.15.1 Confusion matrix

Confusion matrix terdiri atas:

- **True Positive (TP):** kelas aktual Ya diprediksi Ya;
- **True Negative (TN):** kelas aktual Tidak diprediksi Tidak;
- **False Positive (FP):** kelas aktual Tidak diprediksi Ya;
- **False Negative (FN):** kelas aktual Ya diprediksi Tidak.

Struktur confusion matrix ditunjukkan pada Tabel 10.

Tabel 10. Struktur confusion matrix

	Prediksi Tidak	Prediksi Ya
Aktual Tidak	True Negative	False Positive
Aktual Ya	False Negative	True Positive

2.15.2 Accuracy

Accuracy menunjukkan proporsi seluruh prediksi yang benar.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy tetap digunakan, tetapi tidak dijadikan satu-satunya dasar pemilihan model.

2.15.3 Precision

Precision menunjukkan seberapa banyak prediksi kelas positif yang benar-benar termasuk kelas positif.

$$Precision = \frac{TP}{TP + FP}$$

Precision yang tinggi menunjukkan bahwa model memiliki false positive yang relatif rendah.

2.15.4 Recall

Recall menunjukkan seberapa banyak data positif aktual yang berhasil dikenali oleh model.

$$Recall = \frac{TP}{TP + FN}$$

Recall kelas Ya penting dalam target AI Agents karena menunjukkan kemampuan model menemukan developer yang menggunakan AI Agent.

2.15.5 F1-score

F1-score merupakan rata-rata harmonis antara precision dan recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Penelitian menghitung F1-score secara terpisah untuk kelas Tidak dan kelas Ya.

2.15.6 F1 Macro

F1 Macro merupakan rata-rata F1-score seluruh kelas tanpa mempertimbangkan jumlah data pada setiap kelas.

$$F1\ Macro = \frac{F1_{Tidak} + F1_{Ya}}{2}$$

F1 Macro digunakan sebagai metrik utama dalam membandingkan model karena memberikan bobot yang sama kepada kelas Tidak dan kelas Ya. Model dengan F1 Macro lebih tinggi dianggap memiliki performa yang lebih seimbang.

2.15.7 Feature importance

Feature importance digunakan untuk mengetahui fitur yang paling banyak memberikan kontribusi terhadap proses prediksi model. Penelitian menggunakan feature importance dari CatBoost dan Random Forest.

Nilai feature importance yang tinggi menunjukkan bahwa fitur tersebut lebih informatif bagi model dalam membedakan kelas. Namun, feature importance hanya menggambarkan hubungan prediktif dan tidak membuktikan bahwa fitur tersebut menyebabkan seseorang menggunakan AI.

2.16 Kriteria Penentuan Model Terbaik

Model terbaik ditentukan berdasarkan beberapa pertimbangan berikut:

1. Memiliki nilai F1 Macro tertinggi.
2. Memiliki keseimbangan F1-score antara kelas Tidak dan kelas Ya.
3. Memiliki recall kelas Ya yang memadai, terutama pada target AIAgents.
4. Tidak hanya memiliki accuracy tinggi karena dominasi kelas mayoritas.
5. Menghasilkan confusion matrix dengan jumlah kesalahan klasifikasi yang lebih rendah dan lebih seimbang.

Hasil evaluasi setiap skenario, confusion matrix, source code, feature importance, serta perbandingan model dijelaskan pada BAB III.

BAB III

HASIL DAN PEMBAHASAN

3.1 Implementasi Penelitian

Penelitian prediksi adopsi Artificial Intelligence pada developer diimplementasikan menggunakan bahasa pemrograman Python melalui Google Colab. Dataset yang digunakan adalah Stack Overflow Developer Survey 2025 dengan file utama bernama results.csv.

Implementasi penelitian terdiri atas beberapa tahapan, yaitu pemuatan dataset, pemilihan fitur, feature engineering, eksplorasi data, pembagian data training dan testing, preprocessing, penanganan ketidakseimbangan kelas, pelatihan model, evaluasi, hyperparameter tuning, dan interpretasi feature importance.

Penelitian menggunakan dua skenario target klasifikasi, yaitu:

1. AISelect, untuk memprediksi penggunaan AI Tools secara umum.
2. AIAgents, untuk memprediksi penggunaan AI Agent dalam pekerjaan developer.

Model yang dibandingkan adalah Logistic Regression, Random Forest, dan CatBoost. Evaluasi utama menggunakan F1 Macro karena distribusi kelas pada dataset tidak seimbang. Hasil presentasi kelompok juga menunjukkan bahwa CatBoost menjadi model paling konsisten pada kedua target.

3.2 Hasil Pemuatan Dataset

Dataset dimuat dari Google Drive dengan melakukan pemeriksaan terhadap keberadaan file dan seluruh kolom yang diperlukan. Source code pemuatan dataset ditunjukkan sebagai berikut:

```
from google.colab import drive

drive.mount('/content/drive')

PROJECT_DIR = Path(
    '/content/drive/MyDrive/Colab Notebooks/PROJECT ML'
)

candidate_dirs = [
    PROJECT_DIR / 'dataset',
    Path('/content/dataset'),
    Path('/content')
]
```

```

DATA_DIR = next(
    (
        path for path in candidate_dirs
        if (path / 'results.csv').exists()
    ),
    None
)

if DATA_DIR is None:
    raise FileNotFoundError('results.csv tidak ditemukan.')

data_path = DATA_DIR / 'results.csv'

df_raw = pd.read_csv(
    data_path,
    usecols=REQUIRED_COLUMNS,
    low_memory=False
)

print(f'Dataset berhasil dimuat: {df_raw.shape[0]:,} baris')
display(df_raw.head(5))

```

Hasil eksekusi menunjukkan bahwa dataset berhasil dimuat sebanyak:

Dataset berhasil dimuat: 49.191 baris

Dataset awal terdiri atas **49.191 responden**. Dari 172 kolom yang tersedia pada dataset asli, penelitian hanya membaca 13 kolom yang diperlukan, yaitu 11 fitur prediktor dan dua target klasifikasi.

Dataset berhasil dimuat: 49,191 baris

	Age	WorkExp	YearsCode	DevType	CompTotal	LanguageHaveWorkedWith	DatabaseHaveWorkedWith	WebframeHaveWorkedWith	DevEnvsHaveWorkedW
0	25-34 years old	8.0	14.0	Developer, mobile	52800.0	Bash/Shell (all shells);Dart;SQL	Cloud Firestore;PostgreSQL	NaN	Android Studio;Notepad++;Visual Studio;Visual
1	25-34 years old	2.0	10.0	Developer, back-end	90000.0	Java	Dynamodb;MongoDB	Spring Boot	IntelliJ IDEA;PyCharm;Visual Studio Code;Xcode
2	35-44 years old	10.0	12.0	Developer, front-end	2214000.0	Dart;HTML/CSS;JavaScript;TypeScript	MongoDB;MySQL;PostgreSQL	Next.js;Node.js;React	Visual Studio Co

Gambar 6. Hasil pemuatan dan tiga contoh data asli

Pada data asli, masih ditemukan beberapa karakteristik yang belum dapat langsung digunakan oleh model, yaitu:

- usia masih berbentuk rentang;
- beberapa nilai numerik tersimpan sebagai teks;
- terdapat missing value;
- fitur teknologi berisi beberapa jawaban dalam satu sel;
- target masih berbentuk kalimat;
- nilai kompensasi memiliki rentang yang sangat besar.

Oleh karena itu, dataset harus melalui tahap feature engineering dan preprocessing sebelum digunakan dalam pelatihan model.

3.3 Hasil Feature Engineering

3.3.1 Pemilihan fitur

Penelitian menggunakan 11 fitur prediktor sebagai berikut:

```
NUMERIC_FEATURES = [  
    'Age',  
    'YearsCode',  
    'WorkExp',  
    'CompTotal'  
]  
  
CATEGORICAL_FEATURES = [  
    'DevType',  
    'OpSys',  
    'DevEnvs',  
    'Language',  
    'Webframe',  
    'Database',  
    'AISent'  
]
```

Hasil eksekusi notebook menampilkan:

Fitur yang digunakan:

```
['Age', 'YearsCode', 'WorkExp', 'CompTotal',  
 'DevType', 'OpSys', 'DevEnvs', 'Language',  
 'Webframe', 'Database', 'AISent']
```

Pemilihan fitur tersebut didasarkan pada beberapa pertimbangan. Age, YearsCode, dan WorkExp digunakan untuk menggambarkan usia dan tingkat pengalaman developer. CompTotal digunakan sebagai informasi kondisi profesional responden. Sementara itu, DevType, OpSys, DevEnvs, Language, Webframe, dan Database menggambarkan lingkungan teknologi developer.

Fitur AISent digunakan untuk menggambarkan sentimen developer terhadap AI. Fitur ini dinilai relevan karena sikap developer terhadap AI dapat menjadi sinyal yang membantu model membedakan pengguna dan non-pengguna AI.

Pemilihan fitur tidak semata-mata didasarkan pada jumlah kolom yang tersedia, tetapi melalui proses eksplorasi dan percobaan pemodelan. Fitur yang tidak relevan dengan target, memiliki informasi yang terlalu khusus, atau berpotensi menyebabkan data leakage tidak digunakan.

3.3.2 Transformasi usia

Nilai usia yang sebelumnya berbentuk rentang diubah menjadi angka tengah. Source code transformasi usia adalah sebagai berikut:

```
def age_to_number(value):
    if pd.isna(value):
        return np.nan

    text = str(value)

    if 'Under 18' in text:
        return 17

    if '65 years or older' in text:
        return 70

    numbers = [
        float(number)
        for number in re.findall(r'\d+', text)
    ]

    return np.mean(numbers) if numbers else np.nan
```

Contoh hasil transformasi:

Nilai awal	Hasil transformasi
25–34 years old	29,5

35–44 years old	39,5
Under 18 years old	17
65 years or older	70

Transformasi ini mengubah fitur usia menjadi numerik agar dapat diproses oleh model Machine Learning.

3.3.3 Transformasi pengalaman coding

Kolom YearsCode memiliki beberapa jawaban khusus yang tidak dapat langsung dikonversi menjadi angka. Source code yang digunakan adalah:

```
def years_code_to_number(value):
    if pd.isna(value):
        return np.nan

    special_values = {
        'Less than 1 year': 0.5,
        'More than 50 years': 51,
    }

    return pd.to_numeric(
        special_values.get(str(value), value),
        errors='coerce'
    )
```

Jawaban Less than 1 year diubah menjadi 0,5, sedangkan More than 50 years diubah menjadi 51. Nilai lainnya dikonversi langsung menjadi data numerik.

3.3.4 Pembentukan target

Target AISelect dan AIAgents diubah menjadi kelas biner. Source code yang digunakan adalah:

```
def target_to_binary(series):
    text = series.astype('string')

    result = pd.Series(
        np.nan,
        index=series.index,
        dtype='float64'
    )
```

```

result.loc[
    text.str.startswith('Yes', na=False)
] = 1

result.loc[
    text.str.startswith('No', na=False)
] = 0

return result

```

Aturan pembentukan kelas adalah:

Awal jawaban	Kelas
Yes	1
No	0
Kosong atau jawaban lain	Tidak digunakan

Dengan aturan tersebut, variasi jawaban seperti Yes, daily, Yes, weekly, atau Yes, monthly or infrequently tetap dimasukkan ke dalam kelas 1.

3.4 Hasil Analisis Eksploratif

Analisis eksploratif dilakukan terhadap fitur AISent untuk mengetahui distribusi sentimen developer terhadap Artificial Intelligence.

Source code visualisasi adalah sebagai berikut:

```

sentiment_order = X['AISent'].value_counts().index

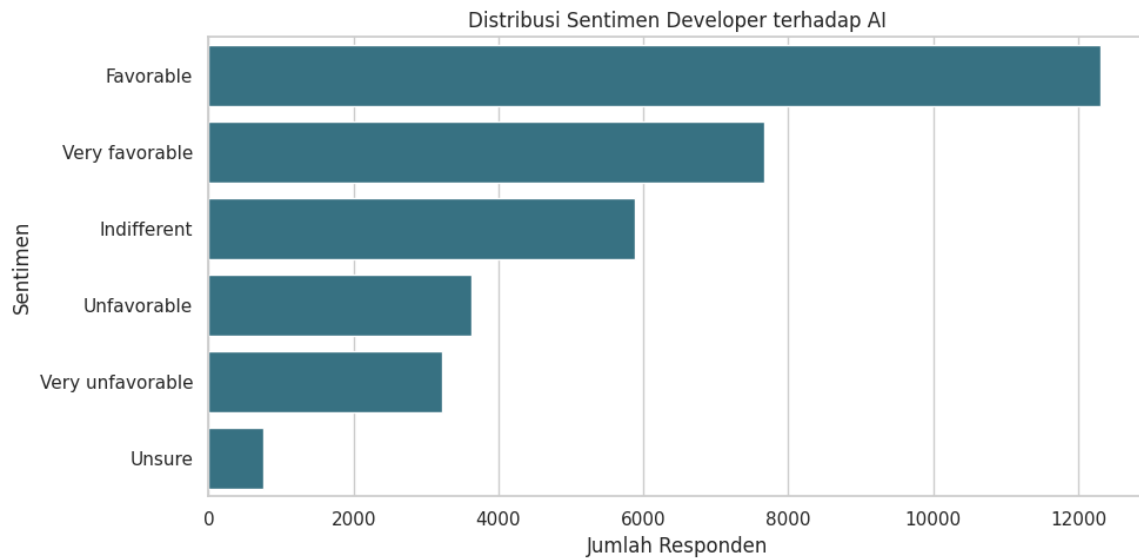
plt.figure(figsize=(10, 5))

sns.countplot(
    data=X,
    y='AISent',
    order=sentiment_order
)

plt.title('Distribusi Sentimen Developer terhadap AI')
plt.xlabel('Jumlah Responden')
plt.ylabel('Sentimen')
plt.tight_layout()

```

```
plt.show()
```



Gambar 7. Distribusi sentimen developer terhadap AI

Berdasarkan grafik tersebut, kategori Favorable memiliki jumlah responden tertinggi, kemudian diikuti oleh Very favorable dan Indifferent. Kategori Unfavorable dan Very unfavorable memiliki jumlah yang lebih rendah, sedangkan kategori Unsure menjadi kategori dengan jumlah paling sedikit.

Distribusi ini menunjukkan bahwa sebagian besar responden memiliki sentimen positif terhadap AI. Meskipun demikian, grafik tersebut hanya menunjukkan distribusi awal dan belum membuktikan bahwa sentimen positif secara langsung menyebabkan penggunaan AI. Pengaruh fitur AISent terhadap prediksi selanjutnya dianalisis melalui feature importance.

3.5 Hasil Skenario A: Prediksi AISelect

3.5.1 Pembagian dan balancing data

Pada target AISelect, hanya data dengan target valid yang digunakan. Data dibagi menjadi 80% training dan 20% testing menggunakan stratifikasi kelas.

Source code pemanggilan model adalah:

```
result_ai_select = train_and_evaluate(  
    X,  
    Y_1,  
    'Prediksi AISelect'  
)
```


Hasil pembagian data adalah:

Training awal : 26.976

Setelah SMOTENC : 42.350

Testing : 6.744

SMOTENC menambah jumlah data kelas minoritas pada training set sehingga jumlah training meningkat dari 26.976 menjadi 42.350 data. Data testing sebanyak 6.744 tetap menggunakan distribusi asli.

3.5.2 Hasil evaluasi model AISelect

Hasil evaluasi tiga model pada target AISelect ditunjukkan pada Tabel 3.11.

Tabel 11. Hasil evaluasi model pada target AISelect

Model	F1 Tidak	F1 Ya	F1 Macro	Accuracy
Logistic Regression	0,6628	0,8856	0,7742	0,8292
Random Forest	0,6712	0,8906	0,7809	0,8359
CatBoost	0,6723	0,9079	0,7901	0,8562

CatBoost memperoleh nilai F1 Macro tertinggi sebesar **0,7901** dan accuracy sebesar **0,8562**. Random Forest berada pada posisi kedua dengan F1 Macro sebesar 0,7809, sedangkan Logistic Regression memperoleh F1 Macro sebesar 0,7742.

Nilai F1 kelas Ya pada seluruh model lebih tinggi daripada F1 kelas Tidak. Hal tersebut menunjukkan bahwa ketiga model lebih mudah mengenali developer yang menggunakan AI Tools dibandingkan developer yang tidak menggunakan AI Tools.

CatBoost menjadi model paling baik karena memberikan nilai tertinggi pada F1 Ya, F1 Macro, dan accuracy. Namun, nilai F1 Tidak sebesar 0,6723 menunjukkan bahwa kemampuan model dalam mengenali non-pengguna AI masih belum sebaik kemampuannya mengenali pengguna AI.

3.5.3 Classification report AISelect

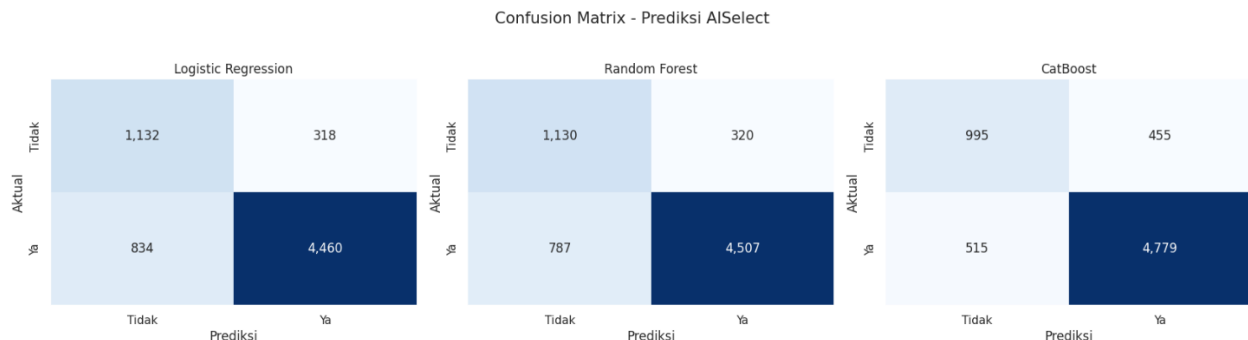
Hasil classification report CatBoost ditunjukkan sebagai berikut:

	precision	recall	f1-score	support
Tidak	0,6589	0,6862	0,6723	1.450
Ya	0,9131	0,9027	0,9079	5.294
Accuracy			0,8562	6.744
Macro Avg	0,7860	0,7945	0,7901	6.744

Precision kelas Ya sebesar 0,9131 menunjukkan bahwa dari seluruh responden yang diprediksi menggunakan AI Tools, sekitar 91,31% benar-benar berasal dari kelas Ya. Recall sebesar 0,9027 menunjukkan bahwa model berhasil mengenali sekitar 90,27% dari seluruh pengguna AI Tools pada data testing.

Sebaliknya, recall kelas Tidak sebesar 0,6862 menunjukkan bahwa masih terdapat sejumlah non-pengguna AI yang salah diklasifikasikan sebagai pengguna.

3.5.4 Confusion matrix AISelect



Gambar 8. Confusion matrix model pada target AISelect

Nilai confusion matrix ditunjukkan pada Tabel 12.

Tabel 12. Confusion matrix AISelect

Model	Benar Tidak	Salah menjadi Ya	Salah menjadi Tidak	Benar Ya
Logistic Regression	1.132	318	834	4.460
Random Forest	1.130	320	787	4.507
CatBoost	995	455	515	4.779

CatBoost berhasil mengenali **4.779 pengguna AI Tools** dengan benar, lebih tinggi daripada Random Forest sebanyak 4.507 dan Logistic Regression sebanyak 4.460.

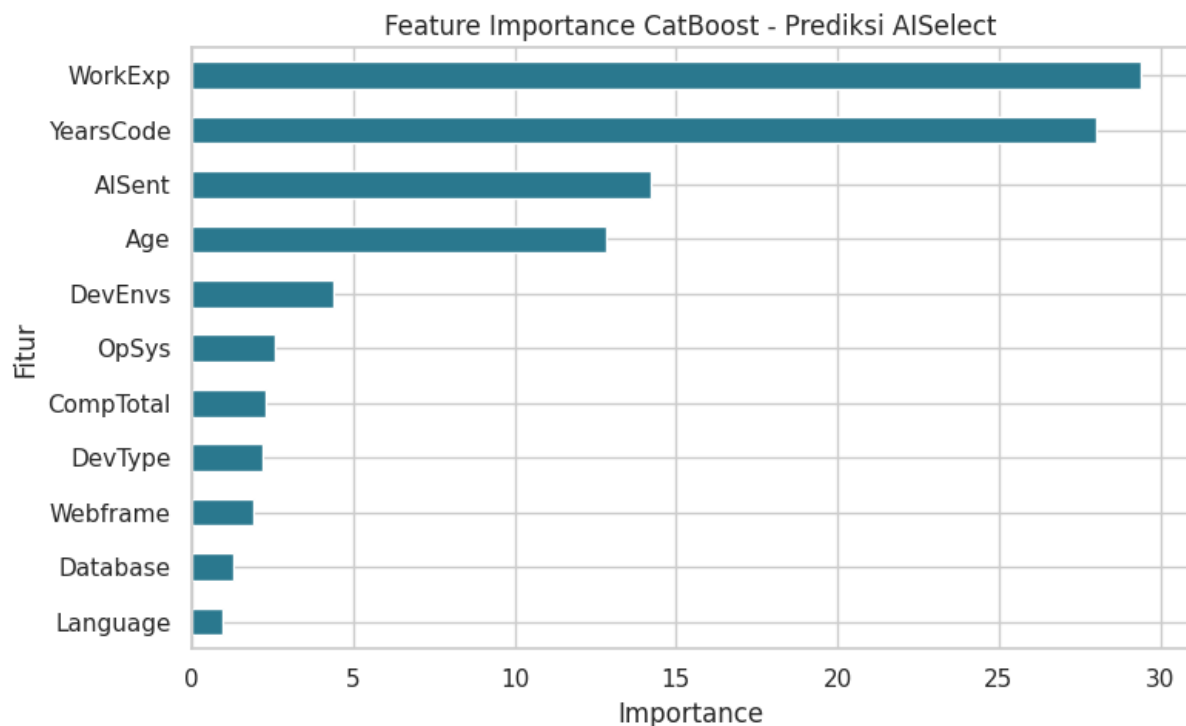
Selain itu, CatBoost hanya salah mengklasifikasikan 515 pengguna AI menjadi kelas Tidak. Jumlah tersebut lebih rendah dibandingkan Random Forest sebanyak 787 dan Logistic Regression sebanyak 834.

Meskipun demikian, CatBoost hanya berhasil mengenali 995 non-pengguna dengan benar dan salah memprediksi 455 non-pengguna sebagai pengguna AI. Hal tersebut menunjukkan adanya trade-off. CatBoost lebih kuat dalam menangkap kelas Ya, tetapi menghasilkan kesalahan lebih besar pada sebagian kelas Tidak.

3.5.5 Feature importance AISelect

Source code untuk menampilkan feature importance CatBoost adalah sebagai berikut:

```
feature_importance = pd.Series(  
    catboost_model.feature_importances_,  
    index=FEATURE_COLUMNS  
)  
  
feature_importance = feature_importance.sort_values()  
  
feature_importance.plot.barh(figsize=(10, 6))  
  
plt.title(  
    'Feature Importance CatBoost - Prediksi AISelect'  
)  
plt.xlabel('Importance')  
plt.ylabel('Fitur')  
plt.tight_layout()  
plt.show()
```



Gambar 9. Feature importance CatBoost pada target AISelect

Feature importance menunjukkan bahwa fitur paling dominan adalah:

1. WorkExp;
2. YearsCode;
3. AISent;
4. Age;
5. DevEnvs.

WorkExp dan YearsCode memiliki kontribusi prediktif paling tinggi. Artinya, pengalaman kerja dan pengalaman coding merupakan informasi yang paling banyak digunakan CatBoost dalam membedakan pengguna dan non-pengguna AI Tools.

AISent berada pada posisi berikutnya. Hasil tersebut menunjukkan bahwa sentimen terhadap AI juga memberikan informasi penting dalam proses prediksi. Developer dengan sentimen berbeda cenderung memiliki pola penggunaan AI yang berbeda pula.

Fitur Language dan Database memiliki importance lebih rendah. Namun, importance yang rendah tidak berarti fitur tersebut tidak memiliki hubungan sama sekali dengan adopsi AI. Informasi fitur tersebut dapat memiliki pola yang tersebar pada kategori lain atau memiliki kontribusi yang lebih kecil setelah dikombinasikan dengan fitur pengalaman dan sentimen.

3.6 Hasil Skenario B: Prediksi AI Agents Awal

3.6.1 Pembagian dan balancing data

Source code pemodelan target AI Agents adalah:

```
result_ai_agents = train_and_evaluate(  
    X,  
    Y_2,  
    'Prediksi AI Agents'  
)
```

Hasil pembagian data adalah:

Training awal : 25.535

Setelah SMOTENC : 35.270

Testing : 6.384

Jumlah data valid target AI Agents lebih sedikit dibandingkan AISelect. Hal ini disebabkan tidak seluruh responden memberikan jawaban Yes atau No yang dapat dibentuk menjadi kelas biner.

3.6.2 Hasil evaluasi model AIAgents awal

Tabel 13. Hasil evaluasi awal model pada target AIAgents

Model	F1 Tidak	F1 Ya	F1 Macro	Accuracy
Logistic Regression	0,7189	0,5377	0,6283	0,6504
Random Forest	0,7269	0,5319	0,6294	0,6551
CatBoost	0,7881	0,4861	0,6371	0,6999

CatBoost memperoleh F1 Macro tertinggi sebesar **0,6371** dan accuracy sebesar **0,6999**. Namun, F1 kelas Ya CatBoost hanya sebesar 0,4861, lebih rendah daripada Logistic Regression dan Random Forest.

Hasil tersebut menunjukkan bahwa CatBoost awal lebih banyak memperoleh performa tinggi karena kemampuannya dalam mengenali kelas Tidak. CatBoost belum cukup baik dalam mengenali developer yang menggunakan AI Agent.

Logistic Regression memiliki F1 Ya tertinggi sebesar 0,5377. Akan tetapi, F1 Tidak dan accuracy model tersebut lebih rendah. Dengan demikian, tidak ada model awal yang sepenuhnya unggul pada seluruh metrik.

3.6.3 Classification report AIAgents

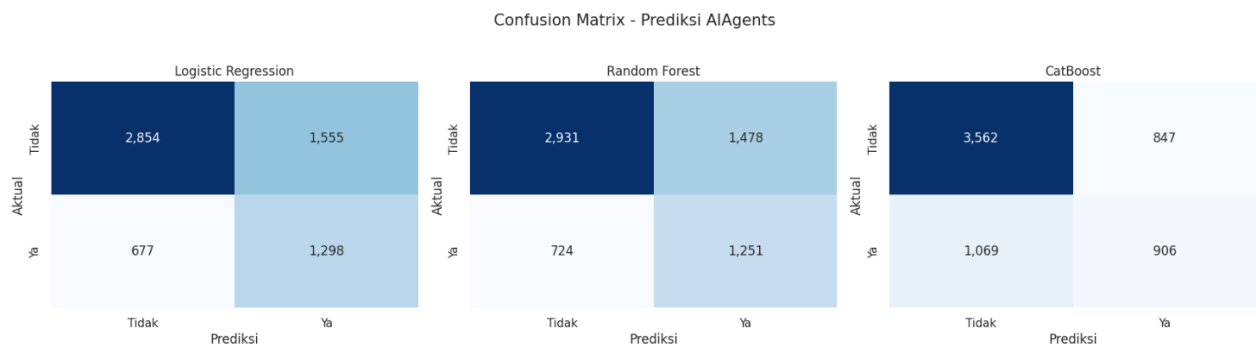
Classification report CatBoost awal adalah:

	precision	recall	f1-score	support
Tidak	0,7692	0,8079	0,7881	4.409
Ya	0,5168	0,4587	0,4861	1.975
Accuracy			0,6999	6.384
Macro Avg	0,6430	0,6333	0,6371	6.384

Recall kelas Ya hanya sebesar 0,4587. Artinya, CatBoost hanya berhasil mengenali sekitar 45,87% dari seluruh pengguna AI Agent pada data testing. Sebaliknya, recall kelas Tidak mencapai 0,8079.

Perbedaan tersebut menunjukkan bahwa model cenderung bias terhadap kelas Tidak. Kondisi ini disebabkan kelas pengguna AI Agent memiliki jumlah lebih sedikit dan pola adopsinya lebih sulit dibandingkan penggunaan AI Tools secara umum.

3.6.4 Confusion matrix AI Agents awal



Gambar 10. Confusion matrix awal pada target AI Agents

Tabel 14. Confusion matrix AI Agents awal

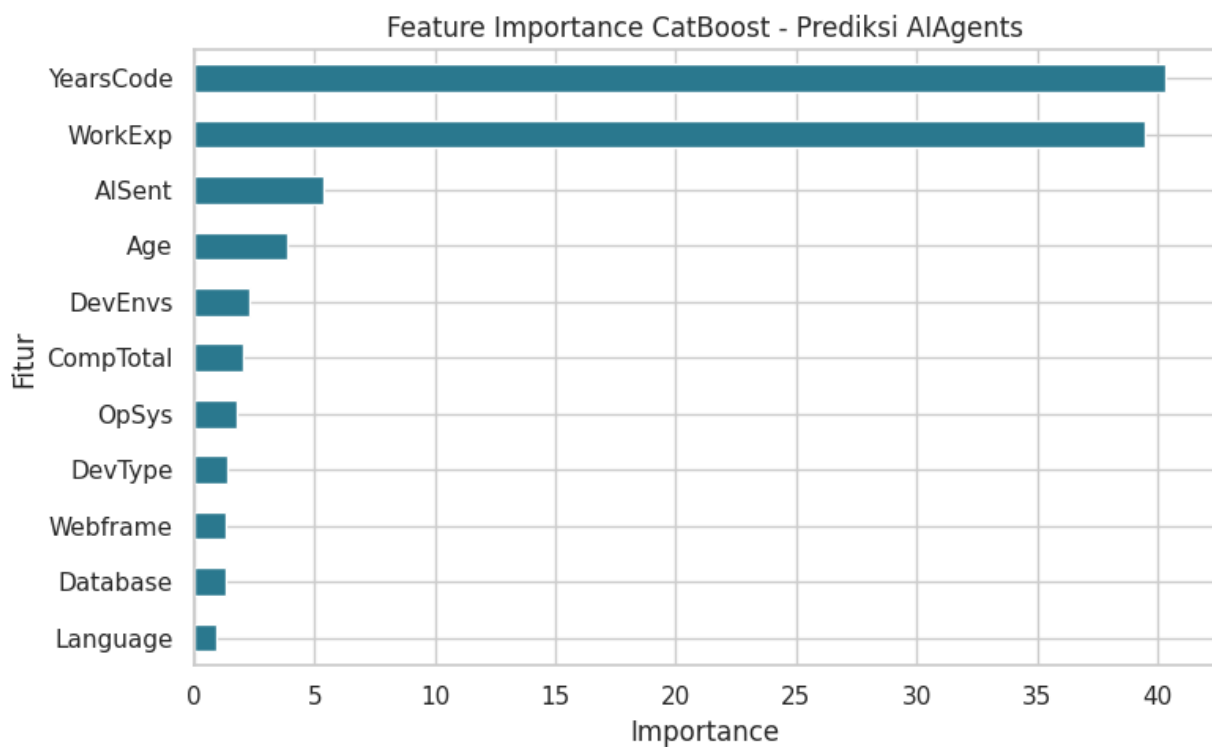
Model	Benar Tidak	Salah menjadi Ya	Salah menjadi Tidak	Benar Ya
Logistic Regression	2.854	1.555	677	1.298
Random Forest	2.931	1.478	724	1.251
CatBoost	3.562	847	1.069	906

CatBoost berhasil mengenali 3.562 non-pengguna AI Agent, lebih tinggi dibandingkan model lainnya. Namun, CatBoost hanya mengenali 906 pengguna AI Agent dengan benar dan salah mengklasifikasikan 1.069 pengguna sebagai kelas Tidak.

Logistic Regression mampu mengenali 1.298 pengguna AI Agent, tetapi menghasilkan 1.555 false positive. Random Forest memiliki pola yang relatif serupa dengan 1.251 true positive dan 1.478 false positive.

Hasil confusion matrix menunjukkan bahwa peningkatan kemampuan mengenali kelas Ya sering diikuti peningkatan kesalahan pada kelas Tidak. Oleh karena itu, optimasi perlu dilakukan dengan mempertimbangkan keseimbangan kedua kelas, bukan hanya accuracy.

3.6.5 Feature importance AI Agents



Gambar 11. Feature importance CatBoost pada target AI Agents

Urutan feature importance utama pada target AI Agents adalah:

1. YearsCode;
2. WorkExp;
3. AISent;
4. Age;
5. DevEnvs.

YearsCode dan WorkExp terlihat sangat dominan dibandingkan fitur lainnya. Hasil ini menunjukkan bahwa pengalaman coding dan pengalaman kerja menjadi sinyal utama yang digunakan model untuk membedakan pengguna dan non-pengguna AI Agent.

Temuan tersebut konsisten dengan target AISelect. Perbedaannya adalah dominasi pengalaman pada AI Agents terlihat lebih besar. Hal ini menunjukkan bahwa penggunaan AI Agent kemungkinan memiliki pola yang lebih khusus dan berkaitan dengan tingkat pengalaman developer.

Namun, hasil tersebut tetap harus dipahami sebagai hubungan prediktif. Penelitian ini tidak membuktikan bahwa pengalaman kerja secara langsung menyebabkan developer menggunakan AI Agent.

3.7 Optimasi Model AI Agents

Optimasi dilakukan karena model awal masih kesulitan mengenali kelas pengguna AI Agent.

Strategi optimasi yang digunakan terdiri atas:

1. class weighting pada CatBoost;
2. random under-sampling;
3. hyperparameter tuning CatBoost;
4. hyperparameter tuning Logistic Regression;
5. hyperparameter tuning Random Forest.

Data testing tidak diseimbangkan sehingga hasil evaluasi tetap mencerminkan kondisi distribusi data asli.

3.7.1 Hasil hyperparameter tuning CatBoost

CatBoost menggunakan `auto_class_weights='Balanced'` untuk memberikan bobot lebih besar pada kelas minoritas.

GridSearchCV menguji parameter berikut:

```
catboost_params = {
    'depth': [4, 6],
    'learning_rate': [0.05, 0.1],
    'l2_leaf_reg': [1, 5]
}

catboost_grid = GridSearchCV(
    estimator=catboost_model,
    param_grid=catboost_params,
    cv=3,
    scoring='f1_macro',
    verbose=1
)
```

Sebanyak delapan kombinasi parameter diuji menggunakan tiga fold cross-validation sehingga dilakukan 24 proses pelatihan.

Hasil tuning menunjukkan parameter terbaik:

`depth` = 6

`l2_leaf_reg` = 1

`learning_rate` = 0,1

Classification report CatBoost tuned adalah:

	precision	recall	f1-score	support
Tidak	0,8250	0,6659	0,7369	4.409
Ya	0,4786	0,6846	0,5633	1.975
Accuracy			0,6717	6.384
Macro Avg	0,6518	0,6752	0,6501	6.384

Setelah optimasi, F1 kelas Ya meningkat dari **0,4861 menjadi 0,5633**. Recall kelas Ya juga meningkat dari 0,4587 menjadi 0,6846. Artinya, kemampuan CatBoost dalam menemukan pengguna AI Agent meningkat secara berarti.

F1 Tidak turun dari 0,7881 menjadi 0,7369 dan accuracy turun dari 0,6999 menjadi 0,6717. Penurunan tersebut terjadi karena model menjadi lebih berani memprediksi kelas Ya. Dalam klasifikasi tidak seimbang, penurunan accuracy tidak selalu menunjukkan model menjadi lebih buruk apabila keseimbangan antarkelas meningkat.

F1 Macro meningkat dari 0,6371 menjadi **0,6501**. Dengan demikian, CatBoost tuned menghasilkan performa yang lebih seimbang dibandingkan CatBoost awal.

3.7.2 Hasil random under-sampling

Random under-sampling mengurangi jumlah data kelas mayoritas pada training set.

Source code yang digunakan adalah:

```
rus = RandomUnderSampler(
    random_state=RANDOM_STATE
)

X_train_res, y_train_res = rus.fit_resample(
    X_train_opt,
    y_train_opt
)

X_test_res = X_test_opt
y_test_res = y_test_opt
```

Hasil proses under-sampling adalah:

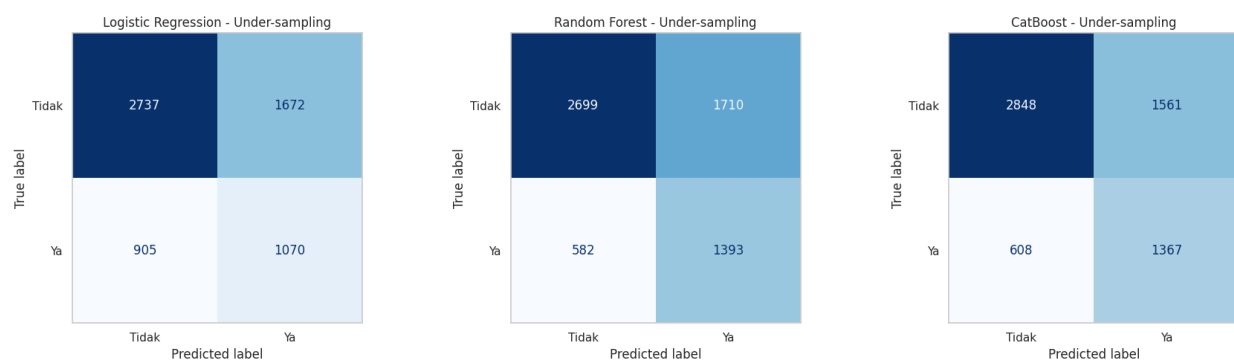
Training sebelum under-sampling	: 25.535
Training setelah under-sampling	: 15.800
Testing dengan distribusi asli	: 6.384

Data training setelah under-sampling memiliki distribusi kelas 50:50. Hasil evaluasi awal setelah under-sampling ditunjukkan pada Tabel 15.

Tabel 15. Hasil model setelah under-sampling

Model	F1 Tidak	F1 Ya	F1 Macro
Logistic Regression	0,6799	0,4537	0,5668
Random Forest	0,7020	0,5486	0,6253
CatBoost	0,7242	0,5576	0,6409

CatBoost kembali menghasilkan F1 Macro tertinggi sebesar 0,6409. F1 Ya CatBoost meningkat dibandingkan model awal, yaitu dari 0,4861 menjadi 0,5576. Namun, F1 Tidak turun dari 0,7881 menjadi 0,7242.



Gambar 12. Confusion matrix model setelah under-sampling

Tabel 16. Confusion matrix setelah under-sampling

Model	Benar Tidak	Salah menjadi Ya	Salah menjadi Tidak	Benar Ya
Logistic Regression	2.737	1.672	905	1.070
Random Forest	2.699	1.710	582	1.393
CatBoost	2.848	1.561	608	1.367

Setelah under-sampling, CatBoost berhasil mengenali 1.367 pengguna AI Agent, meningkat dari CatBoost awal yang hanya mengenali 906 pengguna. Random Forest menghasilkan true positive tertinggi sebanyak 1.393.

Namun, false positive juga meningkat. CatBoost salah memprediksi 1.561 non-pengguna sebagai pengguna AI Agent. Kondisi ini menunjukkan trade-off antara peningkatan sensitivitas terhadap kelas Ya dan penurunan kemampuan mengenali kelas Tidak.

3.7.3 Hasil tuning Logistic Regression

GridSearchCV menguji nilai parameter C sebagai berikut:

```
logreg_params = {  
    'model__C': [0.01, 0.1, 1.0, 10.0],  
    'model__penalty': ['l2']  
}
```

Sebanyak empat kombinasi parameter diuji menggunakan tiga fold cross-validation. Parameter terbaik yang diperoleh adalah:

C = 0,01

penalty = L2

Hasil evaluasi Logistic Regression tuned adalah:

Metrik	Tidak	Ya
Precision	0,7683	0,4142
Recall	0,6391	0,5696
F1-score	0,6978	0,4796

Model memperoleh:

F1 Macro = 0,5887

Accuracy = 0,6176

Hasil tuning meningkatkan F1 Macro Logistic Regression dari 0,5668 pada under-sampling awal menjadi 0,5887. Namun, hasil tersebut masih lebih rendah dibandingkan Random Forest dan CatBoost.

3.7.4 Hasil tuning Random Forest

GridSearchCV menguji parameter berikut:

```
rf_params = {  
    'model__n_estimators': [100, 150],  
    'model__max_depth': [10, 15, 20],  
    'model__min_samples_leaf': [2, 5]  
}
```

Sebanyak 12 kombinasi parameter diuji menggunakan tiga fold cross-validation sehingga dilakukan 36 proses pelatihan.

Parameter terbaik yang diperoleh adalah:

n_estimators = 150

max_depth = 20

min_samples_leaf = 2

Hasil evaluasi Random Forest tuned adalah:

Metrik	Tidak	Ya
Precision	0,8219	0,4529
Recall	0,6217	0,6992
F1-score	0,7079	0,5498

Model memperoleh:

F1 Macro = 0,6288

Accuracy = 0,6457

Recall kelas Ya sebesar 0,6992 menunjukkan bahwa Random Forest tuned mampu mengenali sekitar 69,92% pengguna AI Agent. Namun, F1 Macro model masih berada di bawah CatBoost tuned.

3.8 Perbandingan Hasil Optimasi

Hasil akhir optimasi target AIAgents ditunjukkan pada Tabel 17.

Tabel 17. Perbandingan model hasil optimasi AIAgents

Model	F1 Tidak	F1 Ya	F1 Macro	Accuracy
CatBoost Class Weight + Tuned	0,7369	0,5633	0,6501	0,6717
Random Forest Under-sampling + Tuned	0,7079	0,5498	0,6288	0,6457
Logistic Regression Under-sampling + Tuned	0,6978	0,4796	0,5887	0,6176

CatBoost tuned memiliki F1 Macro tertinggi sebesar 0,6501. Model tersebut juga memiliki F1 Ya tertinggi sebesar 0,5633 dan F1 Tidak tertinggi sebesar 0,7369 di antara model hasil tuning. Random Forest tuned menghasilkan recall kelas Ya yang sedikit lebih tinggi, tetapi precision kelas Ya lebih rendah sehingga F1 Ya berada di bawah CatBoost.

Logistic Regression tuned memiliki performa paling rendah. Hal ini menunjukkan bahwa hubungan antara karakteristik developer dan penggunaan AI Agent kemungkinan tidak sepenuhnya linear. Model berbasis pohon lebih mampu membaca interaksi dan pola kompleks antarfitur.

3.9 Feature Importance Random Forest Tuned

Source code feature importance Random Forest adalah:

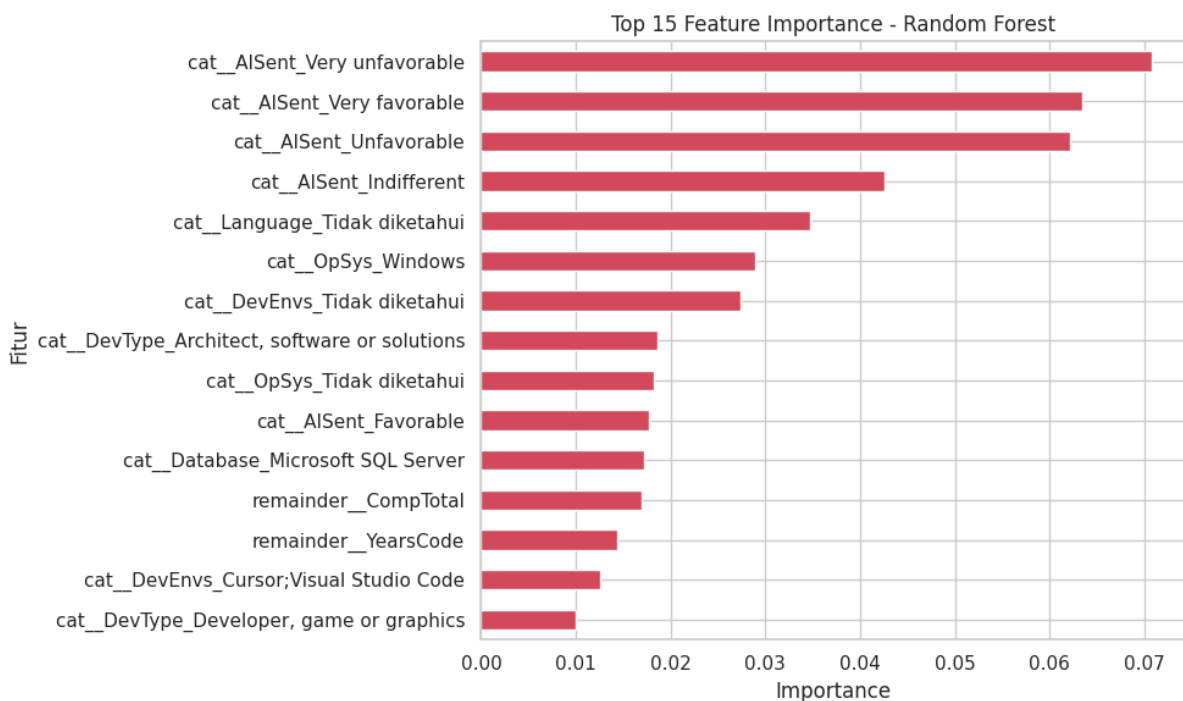
```
best_rf_pipeline = rf_grid.best_estimator_  
  
best_rf = best_rf_pipeline.named_steps['model']  
  
feature_names = (  
    best_rf_pipeline  
    .named_steps['preprocessor']  
    .get_feature_names_out()  
)  
  
rf_importance = pd.Series(  
    best_rf.feature_importances_,  
    index=feature_names  
)  
  
top_rf_importance = (  
    best_rf.feature_importances_.sort_values(ascending=False).head(10).index  
)
```

```

    rf_importance
    .nlargest(15)
    .sort_values()
)

top_rf_importance.plot.barh(figsize=(10, 6))
plt.title('Top 15 Feature Importance - Random Forest')
plt.xlabel('Importance')
plt.ylabel('Fitur')
plt.tight_layout()
plt.show()

```



Gambar 13. Top 15 feature importance Random Forest tuned

Feature importance Random Forest tuned menunjukkan bahwa beberapa kategori AISent berada pada posisi teratas, yaitu:

- Very unfavorable;
- Very favorable;
- Unfavorable;
- Indifferent;
- Favorable.

Fitur lain yang muncul pada 15 posisi teratas adalah:

- bahasa yang tidak diketahui;
- sistem operasi Windows;
- lingkungan pengembangan yang tidak diketahui;
- peran software atau solutions architect;
- sistem operasi yang tidak diketahui;
- Microsoft SQL Server;
- CompTotal;
- YearsCode;
- Cursor dan Visual Studio Code;
- developer game atau graphics.

Hasil tersebut menunjukkan bahwa sentimen terhadap AI menjadi sinyal prediktif yang kuat pada Random Forest tuned. Perbedaan urutan feature importance antara Random Forest dan CatBoost merupakan hal yang wajar karena kedua algoritma memiliki mekanisme pembelajaran yang berbeda.

CatBoost awal lebih menekankan pengalaman kerja dan coding, sedangkan Random Forest tuned lebih banyak menggunakan kategori tertentu dari sentimen AI dalam pemisahan decision tree.

3.10 Perbandingan Seluruh Skenario

Tabel 18. Perbandingan seluruh hasil penelitian

Skenario	Model	F1 Tidak	F1 Ya	F1 Macro
AISelect awal	Logistic Regression	0,6628	0,8856	0,7742
AISelect awal	Random Forest	0,6712	0,8906	0,7809
AISelect awal	CatBoost	0,6723	0,9079	0,7901
AIAgents awal	Logistic Regression	0,7189	0,5377	0,6283
AIAgents awal	Random Forest	0,7269	0,5319	0,6294
AIAgents awal	CatBoost	0,7881	0,4861	0,6371
AIAgents optimasi	Logistic Regression tuned	0,6978	0,4796	0,5887
AIAgents optimasi	Random Forest tuned	0,7079	0,5498	0,6288
AIAgents optimasi	CatBoost tuned	0,7369	0,5633	0,6501

Model terbaik setiap skenario ditunjukkan pada Tabel 19.

Tabel 19. Model terbaik setiap skenario

Skenario	Model terbaik	F1 Macro
AISelect awal	CatBoost	0,7901
AIAgents awal	CatBoost	0,6371
AIAgents optimasi	CatBoost Class Weight + Tuned	0,6501

CatBoost menjadi model terbaik pada seluruh skenario. Hal ini menunjukkan bahwa CatBoost paling konsisten dalam mengolah data survei yang terdiri atas kombinasi fitur numerik dan kategorikal.

Target AISelect menghasilkan F1 Macro sebesar 0,7901, jauh lebih tinggi daripada target AIAgents. Artinya, penggunaan AI Tools secara umum lebih mudah diprediksi dibandingkan penggunaan AI Agent.

Penggunaan AI Tools telah lebih luas sehingga pola pengguna dan non-penggunanya lebih mudah dipelajari. Sebaliknya, penggunaan AI Agent masih lebih spesifik, jumlah penggunanya lebih sedikit, dan kemungkinan dipengaruhi oleh faktor yang belum seluruhnya tersedia dalam dataset.

3.11 Pembahasan Pemilihan Fitur

Sebanyak 11 fitur digunakan setelah dilakukan seleksi berdasarkan relevansi terhadap target dan percobaan model. Pemilihan ini dilakukan agar model tidak menggunakan seluruh 172 kolom tanpa pertimbangan.

Hasil feature importance mendukung penggunaan fitur-fitur tersebut. Pada CatBoost, WorkExp, YearsCode, AISent, dan Age secara konsisten berada pada posisi atas. Pada Random Forest tuned, beberapa kategori AISent juga menjadi fitur yang paling dominan.

Fitur pengalaman memiliki kontribusi tinggi karena pengalaman dapat menggambarkan lama keterlibatan developer dalam lingkungan teknologi. Developer dengan tingkat pengalaman berbeda dapat memiliki pola adaptasi, kebutuhan otomatisasi, dan keterpaparan terhadap AI yang berbeda.

Fitur sentimen memberikan informasi mengenai penerimaan responden terhadap AI. Developer yang memiliki pandangan favorable atau unfavorable dapat menunjukkan kecenderungan penggunaan yang berbeda.

Fitur teknologi seperti sistem operasi, bahasa pemrograman, database, framework, dan lingkungan pengembangan tetap digunakan karena menggambarkan konteks kerja developer.

Meskipun importance beberapa fitur lebih rendah, fitur tersebut dapat membantu model membaca kombinasi karakteristik yang tidak dapat dijelaskan oleh satu fitur secara terpisah. Hasil ini menjawab alasan diperolehnya 11 fitur prediktor. Fitur tidak dipilih secara acak, tetapi berdasarkan:

1. keterkaitan dengan tujuan prediksi;
2. kondisi dan kelengkapan data;
3. kemungkinan fitur diproses oleh model;
4. pencegahan data leakage;
5. hasil eksplorasi;
6. percobaan pemodelan;
7. feature importance yang dihasilkan model.

3.12 Pembahasan Umum

Berdasarkan seluruh eksperimen, terdapat tiga temuan utama.

Pertama, **CatBoost merupakan model paling konsisten**. Model ini memiliki kemampuan yang baik dalam mengolah kombinasi fitur numerik dan kategorikal tanpa bergantung sepenuhnya pada hubungan linear.

Kedua, **AISelect lebih mudah diprediksi daripada AIAgents**. F1 Macro AISelect mencapai 0,7901, sedangkan AIAgents awal hanya mencapai 0,6371. Perbedaan tersebut menunjukkan bahwa adopsi AI Agent memiliki pola yang lebih sulit dan lebih spesifik.

Ketiga, **optimasi berhasil meningkatkan kemampuan model mengenali kelas minoritas**. Pada CatBoost, F1 Ya meningkat dari 0,4861 menjadi 0,5633 dan F1 Macro meningkat dari 0,6371 menjadi 0,6501.

Walaupun accuracy CatBoost turun dari 0,6999 menjadi 0,6717, hasil optimasi tetap dianggap lebih baik untuk tujuan klasifikasi tidak seimbang. Model yang hanya mengejar accuracy dapat lebih sering memilih kelas mayoritas dan gagal mengenali pengguna AI Agent.

Pemilihan model terbaik karena itu tidak hanya berdasarkan accuracy. F1 Macro digunakan sebagai indikator utama karena memberikan bobot yang sama terhadap kelas Tidak dan kelas Ya.

Secara keseluruhan, data Stack Overflow Developer Survey 2025 dapat digunakan untuk memprediksi adopsi AI pada developer. Pengalaman kerja, pengalaman coding, usia, dan sentimen terhadap AI menjadi fitur yang paling informatif dalam proses prediksi. Namun, hasil tersebut merupakan hubungan prediktif dan tidak dapat digunakan untuk menyimpulkan hubungan sebab-akibat.

BAB IV

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Berdasarkan proses pengolahan data, pelatihan model, evaluasi, dan optimasi yang telah dilakukan terhadap dataset **Stack Overflow Developer Survey 2025**, dapat diperoleh kesimpulan sebagai berikut:

1. Dataset Stack Overflow Developer Survey 2025 dapat digunakan untuk membangun model prediksi adopsi Artificial Intelligence pada developer. Dataset awal terdiri atas **49.191 responden dan 172 kolom**, kemudian dipilih 11 fitur prediktor yang berkaitan dengan demografi, pengalaman, kondisi profesional, lingkungan teknologi, dan sentimen terhadap AI. Fitur tersebut meliputi Age, YearsCode, WorkExp, CompTotal, DevType, OpSys, DevEnvs, Language, Webframe, Database, dan AISent.
2. Proses preprocessing diperlukan karena data survei memiliki bentuk yang beragam, seperti rentang usia, nilai numerik yang tersimpan sebagai teks, missing value, data kategorikal, nilai kompensasi ekstrem, dan jawaban multi-value. Proses yang dilakukan meliputi transformasi umur, konversi pengalaman coding, imputasi missing value, clipping nilai ekstrem, transformasi log1p, encoding fitur kategorikal, standardisasi, pembentukan target biner, serta pembagian data training dan testing dengan rasio 80:20.
3. Pada target AISelect, CatBoost menghasilkan performa terbaik dengan nilai **F1 kelas Tidak sebesar 0,6723, F1 kelas Ya sebesar 0,9079, F1 Macro sebesar 0,7901, dan accuracy sebesar 0,8562**. Random Forest memperoleh F1 Macro sebesar 0,7809, sedangkan Logistic Regression memperoleh F1 Macro sebesar 0,7742. Hasil ini menunjukkan bahwa penggunaan AI Tools secara umum relatif lebih mudah diprediksi berdasarkan karakteristik developer.
4. Pada target AIAgents sebelum optimasi, CatBoost memperoleh F1 Macro tertinggi sebesar **0,6371** dan accuracy sebesar **0,6999**. Namun, F1 kelas Ya hanya sebesar 0,4861. Hasil tersebut menunjukkan bahwa model lebih mudah mengenali developer yang tidak menggunakan AI Agent daripada developer yang menggunakannya. Target AIAgents lebih sulit diprediksi karena penggunaan AI Agent masih lebih spesifik dan distribusi kelas pengguna lebih kecil dibandingkan kelas non-pengguna.
5. Optimasi target AIAgents menggunakan class weighting, random under-sampling, dan hyperparameter tuning berhasil meningkatkan keseimbangan performa model. CatBoost tuned dengan parameter `depth = 6`, `learning_rate = 0,1`, dan `l2_leaf_reg = 1`

menghasilkan **F1 kelas Tidak sebesar 0,7369, F1 kelas Ya sebesar 0,5633, F1 Macro sebesar 0,6501, dan accuracy sebesar 0,6717**. F1 kelas Ya meningkat dari 0,4861 menjadi 0,5633, sedangkan F1 Macro meningkat dari 0,6371 menjadi 0,6501.

6. Penurunan accuracy CatBoost setelah optimasi dari 0,6999 menjadi 0,6717 tidak menunjukkan bahwa model menjadi lebih buruk. Model menjadi lebih mampu mengenali kelas pengguna AI Agent, meskipun menghasilkan lebih banyak kesalahan pada kelas non-pengguna. Pada data tidak seimbang, peningkatan F1 Macro dan F1 kelas minoritas lebih penting dibandingkan hanya mempertahankan accuracy yang tinggi.
7. CatBoost menjadi model paling konsisten karena memperoleh F1 Macro tertinggi pada seluruh skenario penelitian. Model ini lebih sesuai untuk dataset survei yang memiliki kombinasi fitur numerik dan kategorikal serta hubungan antarfitur yang kompleks. Random Forest berada pada posisi kedua, sedangkan Logistic Regression menunjukkan performa paling rendah pada target AIAgents setelah optimasi.
8. Indikator utama yang digunakan untuk menentukan model baik atau tidak baik adalah F1 Macro, F1-score setiap kelas, precision, recall, accuracy, dan confusion matrix. Model dinilai lebih baik apabila memiliki F1 Macro tinggi, mampu mengenali kedua kelas secara seimbang, memiliki recall kelas Ya yang memadai, serta tidak hanya memperoleh accuracy tinggi karena dominasi kelas mayoritas. Ketentuan UAS juga menekankan bahwa kesimpulan harus menjelaskan hasil evaluasi, indikator performa, dan model terbaik.
9. Feature importance menunjukkan bahwa WorkExp, YearsCode, AISent, dan Age menjadi fitur yang paling informatif dalam proses prediksi. Pengalaman kerja, pengalaman coding, dan sentimen terhadap AI membantu model membedakan developer pengguna dan non-pengguna AI. Pada Random Forest tuned, beberapa kategori AISent juga muncul sebagai fitur dominan.
10. Hasil feature importance hanya menunjukkan kontribusi prediktif fitur terhadap keputusan model. Hasil tersebut tidak membuktikan bahwa pengalaman, usia, atau sentimen secara langsung menyebabkan developer menggunakan AI. Dengan demikian, penelitian ini menghasilkan kesimpulan prediktif dan bukan kesimpulan sebab-akibat.

Secara keseluruhan, penelitian menunjukkan bahwa karakteristik developer dalam Stack Overflow Developer Survey 2025 dapat digunakan untuk memprediksi adopsi Artificial Intelligence. Penggunaan AI Tools secara umum lebih mudah diprediksi dibandingkan

penggunaan AI Agent. Dari ketiga metode yang dibandingkan, **CatBoost merupakan model terbaik dan paling konsisten**, dengan F1 Macro sebesar 0,7901 pada target AISelect dan 0,6501 pada target AIAgents setelah optimasi.

4.2 Saran

Berdasarkan hasil penelitian dan keterbatasan yang ditemukan, beberapa saran pengembangan yang dapat dilakukan adalah sebagai berikut:

1. Penelitian selanjutnya dapat menggunakan teknik interpretasi model yang lebih mendalam, seperti **SHAP**, untuk mengetahui arah dan besarnya pengaruh setiap fitur terhadap prediksi masing-masing responden. Feature importance yang digunakan dalam penelitian ini hanya menunjukkan tingkat kontribusi fitur secara umum.
2. Hyperparameter tuning dapat dikembangkan dengan ruang pencarian yang lebih luas. Penelitian berikutnya dapat menggunakan RandomizedSearchCV, Bayesian Optimization, atau Optuna agar proses pencarian parameter lebih efisien dan memiliki peluang menemukan kombinasi parameter yang lebih optimal.
3. Penelitian selanjutnya dapat mencoba metode klasifikasi lain, seperti XGBoost, LightGBM, Support Vector Machine, atau neural network. Hasilnya dapat dibandingkan dengan Logistic Regression, Random Forest, dan CatBoost untuk mengetahui apakah terdapat model yang lebih mampu mengenali kelas pengguna AI Agent.
4. Penanganan ketidakseimbangan kelas dapat dikembangkan menggunakan metode lain, seperti Borderline-SMOTE, SMOTE-Tomek, SMOTE-ENN, threshold adjustment, atau cost-sensitive learning. Penggunaan beberapa teknik tersebut dapat membantu meningkatkan recall dan F1-score kelas pengguna AI Agent tanpa menurunkan performa kelas non-pengguna secara terlalu besar.
5. Fitur multi-value seperti Language, Database, Webframe, dan DevEnvs dapat diolah menggunakan pendekatan multi-hot encoding atau representasi fitur yang lebih lengkap. Pemadatan menjadi satu kategori utama dapat mengurangi sebagian informasi mengenai kombinasi teknologi yang digunakan oleh responden.
6. Fitur CompTotal perlu diproses lebih lanjut dengan mempertimbangkan negara, mata uang, dan perbedaan daya beli. Penggunaan total kompensasi secara langsung berpotensi menghasilkan perbandingan yang kurang setara karena responden berasal dari berbagai negara.

7. Analisis dapat dilakukan secara terpisah berdasarkan negara, wilayah, jenis developer, tingkat pengalaman, atau jenis organisasi. Pemisahan kelompok tersebut dapat memberikan pemahaman yang lebih spesifik mengenai karakteristik adopsi AI pada masing-masing segmen developer.
8. Penelitian berikutnya dapat menambahkan fitur yang lebih berkaitan langsung dengan lingkungan kerja, seperti ukuran perusahaan, kebijakan penggunaan AI, akses terhadap AI berbayar, jenis proyek, kebutuhan otomatisasi, dan dukungan organisasi. Fitur tersebut kemungkinan dapat membantu meningkatkan kemampuan model dalam memprediksi penggunaan AI Agent.
9. Model perlu diuji menggunakan dataset Stack Overflow Developer Survey pada tahun berikutnya. Pengujian lintas tahun dapat digunakan untuk mengetahui apakah pola prediksi tetap stabil atau berubah mengikuti perkembangan teknologi AI.
10. Hasil penelitian dapat dikembangkan menjadi prototipe sistem prediksi sederhana. Namun, penggunaannya harus tetap memperhatikan privasi, transparansi, dan potensi bias. Sistem tidak seharusnya digunakan untuk mengambil keputusan terhadap individu tanpa evaluasi manusia dan validasi tambahan.

Dengan pengembangan tersebut, penelitian berikutnya diharapkan dapat menghasilkan model yang lebih akurat, seimbang, mudah diinterpretasikan, dan mampu menggambarkan perkembangan adopsi AI pada developer secara lebih menyeluruh.

DAFTAR PUSTAKA

- [1] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, and A. E. Hassan, “An empirical study of testing practices in open source AI agent frameworks and agentic applications,” *Empir. Softw. Eng.*, vol. 31, no. 5, 2026, doi: 10.1007/s10664-026-10857-9.
- [2] A. S. Yuksel, “LLM verification-evasion patterns in AI-assisted software development: A taxonomy and implications for developer trust,” *Inf. Softw. Technol.*, vol. 199, 2026, doi: 10.1016/j.infsof.2026.108251.
- [3] M. Gemborn Nilsson and B. Bernhardsson, “Classification of multivariate signals using the Hilbert transform and Riemannian geometry,” *Signal Processing*, vol. 250, 2027, doi: 10.1016/j.sigpro.2026.110786.
- [4] F.-S. Zhou *et al.*, “Deep reinforcement learning–driven space decomposition variable neighborhood search algorithm for distributed heterogeneous factory lot-sizing and scheduling problem,” *Expert Syst. Appl.*, vol. 332, 2027, doi: 10.1016/j.eswa.2026.133570.
- [5] S. Aluri and S. S. Imambi, “An Empirical Study of Brain Tumor Classification Using MRI Images,” *Int. J. Image Graph.*, vol. 27, no. 1, 2027, doi: 10.1142/S0219467827500380.
- [6] D. Huang, J. Wan, Z. Zhang, H. Zhang, and C. Wang, “Cluster-guided adversarial graph contrastive learning,” *Neural Networks*, vol. 205, 2027, doi: 10.1016/j.neunet.2026.109315.
- [7] L. Zhu, X. Zhang, C. Zhao, M. Gao, H. Lyu, and Z. Wang, “Prediction of the spatial distribution of settlements in Henan: A grid-based method integrating geographic features and machine learning,” *Expert Syst. Appl.*, vol. 332, 2027, doi: 10.1016/j.eswa.2026.133476.
- [8] D. K. Hein, J. S. Persson, V. V. Jensen, A. R. Bruun, and M. G. Jaatun, “Causal mapping of the risks of using generative AI in software development,” *Inf. Softw. Technol.*, vol. 197, 2026, doi: 10.1016/j.infsof.2026.108202.
- [9] T. Xiao *et al.*, “Self-Admitted GenAI Usage in Open-Source Software,” *IEEE Trans. Softw. Eng.*, vol. 52, no. 6, pp. 1891–1910, 2026, doi: 10.1109/TSE.2026.3681886.
- [10] S. Roy, B. Roy, C. K. Roy, and F. Khomh, “A Qualitative Study on XAI Techniques for Software Defect Prediction,” *Inf. Softw. Technol.*, vol. 198, 2026, doi: 10.1016/j.infsof.2026.108234.